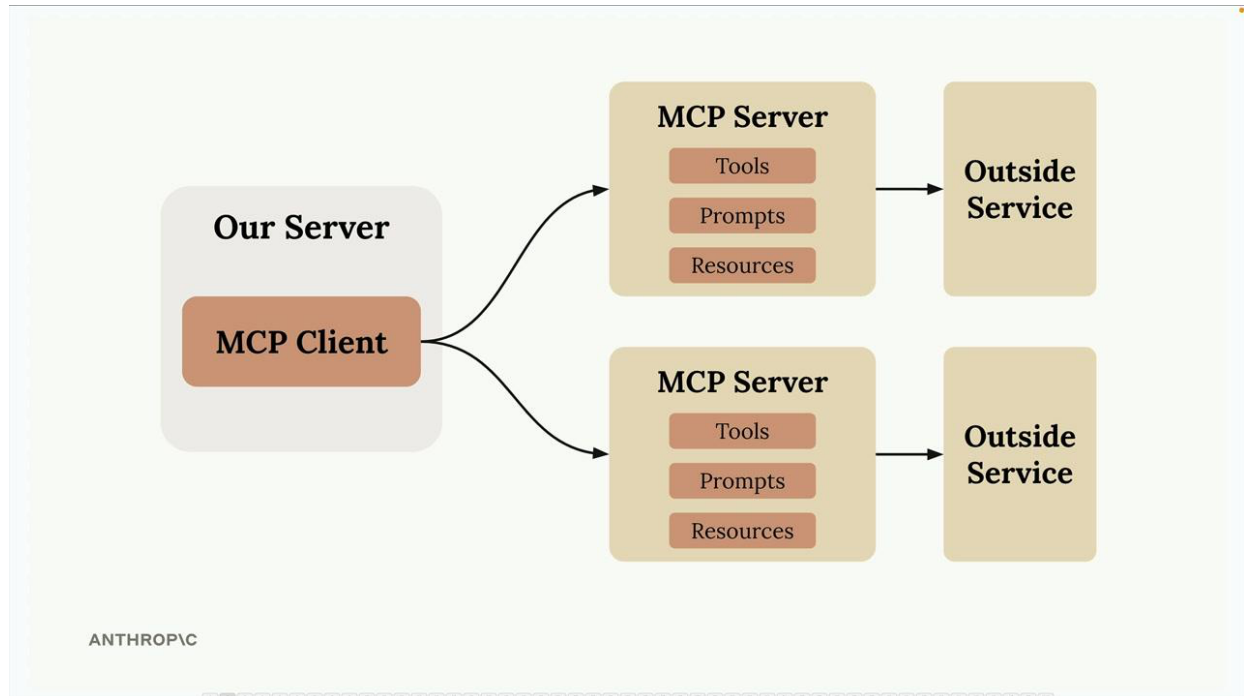


Вступ до МСР

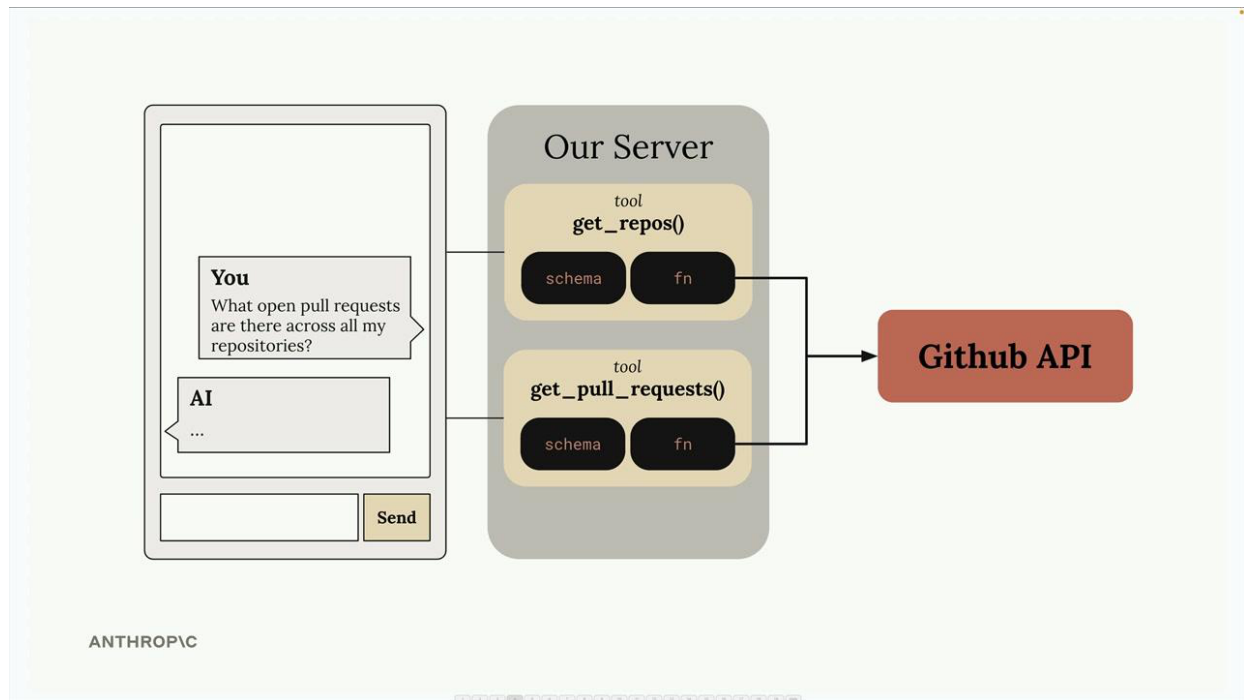
Model Context Protocol (MCP) - це communication layer, який надає Claude context і tools без потреби писати багато нудного integration code. Думайте про нього як про спосіб перенести навантаження з визначення та виконання tools з вашого server на спеціалізовані MCP servers.



Коли ви вперше стикаєтеся з MCP, ви побачите діаграми з базовою architecture: MCP Client (ваш server) підключається до MCP Servers, які містять tools, prompts і resources. Кожен MCP server діє як interface до певного зовнішнього service.

Розуміння MCP на реальному прикладі

Уявімо, що ви створюєте chat interface, де користувачі можуть запитувати Claude про свої GitHub data. Користувач може спитати: "What open pull requests are there across all my repositories?" Щоб відповісти на це, Claude потрібні tools для доступу до GitHub API.



Без MCP вам довелося б самостійно створювати всі GitHub integration tools. Це означає написання schemas і functions для кожної частини GitHub functionality, яку ви хочете підтримувати.

Проблема tool functions

GitHub має величезну functionality - repositories, pull requests, issues, projects і багато іншого. Щоб створити повноцінний GitHub chatbot, вам потрібно було б написати неймовірну кількість tools:

Кожен tool потребує і schema definition, і function implementation. Для developer це означає багато code, який потрібно написати, протестувати та підтримувати.

Tool Functions

- To handle all of Github's functionality, we'd have to create an incredible number of tool schemas and functions
- **This is all code that we (developers) have to write, test, and maintain**

ANTHROPIC

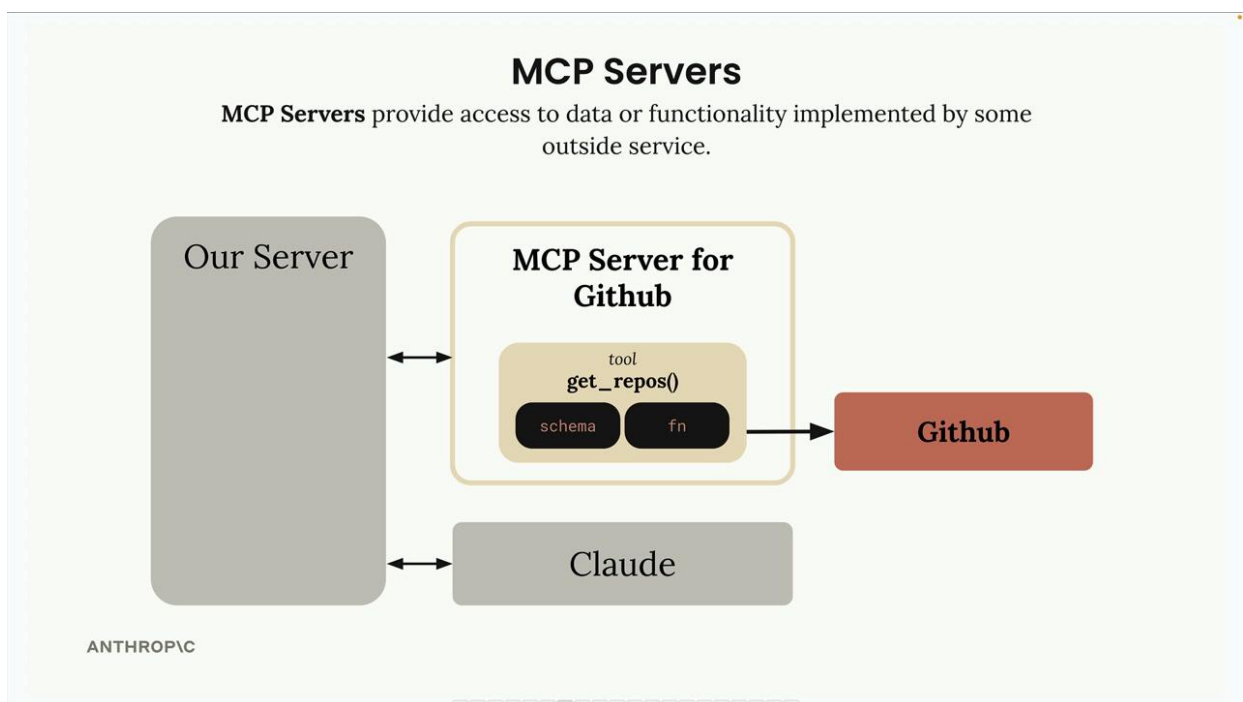
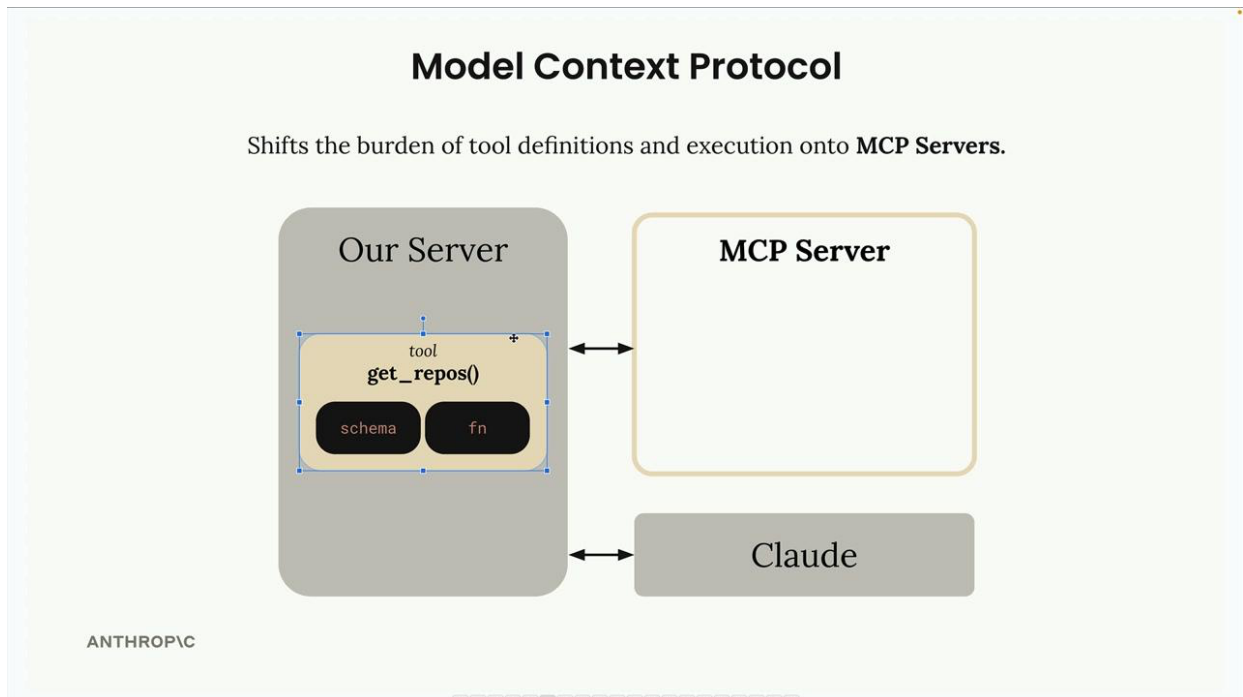
Our Server

tool get_repos() schema fn	tool list_repos() schema fn
tool create_repos() schema fn	tool search_issues() schema fn
tool update_issue() schema fn	tool create_issue() schema fn
tool get_issue() schema fn	tool create_file() schema fn

Як MCP вирішує цю проблему

MCP переносить навантаження з визначення та виконання tools з вашого server на MCP servers. Замість того щоб писати всі ці GitHub tools самостійно, вони створюються й виконуються всередині спеціального MCP server.

MCP server працює як wrapper навколо functionality GitHub, надаючи готові tools, які можна використовувати без потреби реалізовувати їх самостійно.



MCP servers надають доступ до data або functionality, реалізованих зовнішніми services. Вони пакують складні integrations у reusable components, до яких може підключитися будь-який application.

Поширені запитання про MCP

Common Questions

Who authors the MCP Server?	Anyone! Often the service provider itself will make their own MCP implementation. You can make a MCP server to wrap up access to some service.
How is using an MCP Server different from just calling a service's API directly?	MCP Servers provide tool schemas + functions. If you want to directly call an API directly, you'll be authoring those on your own.
Sounds like MCP Servers and tool use are the same thing.	MCP Servers provide tool schemas + functions already defined for you.

ANTHROPIC

Хто створює MCP Servers?

Будь-хто може створити implementation MCP server. Часто самі service providers випускають власні офіційні MCP implementations. Наприклад, AWS може випустити офіційний MCP server з tools для своїх різних services.

Чим MCP відрізняється від direct API calls?

MCP servers надають tool schemas і functions, уже визначені для вас. Якщо ви викликаєте API напрямку, ви самі відповідаєте за створення цих tool definitions. MCP економить вам цю implementation work.

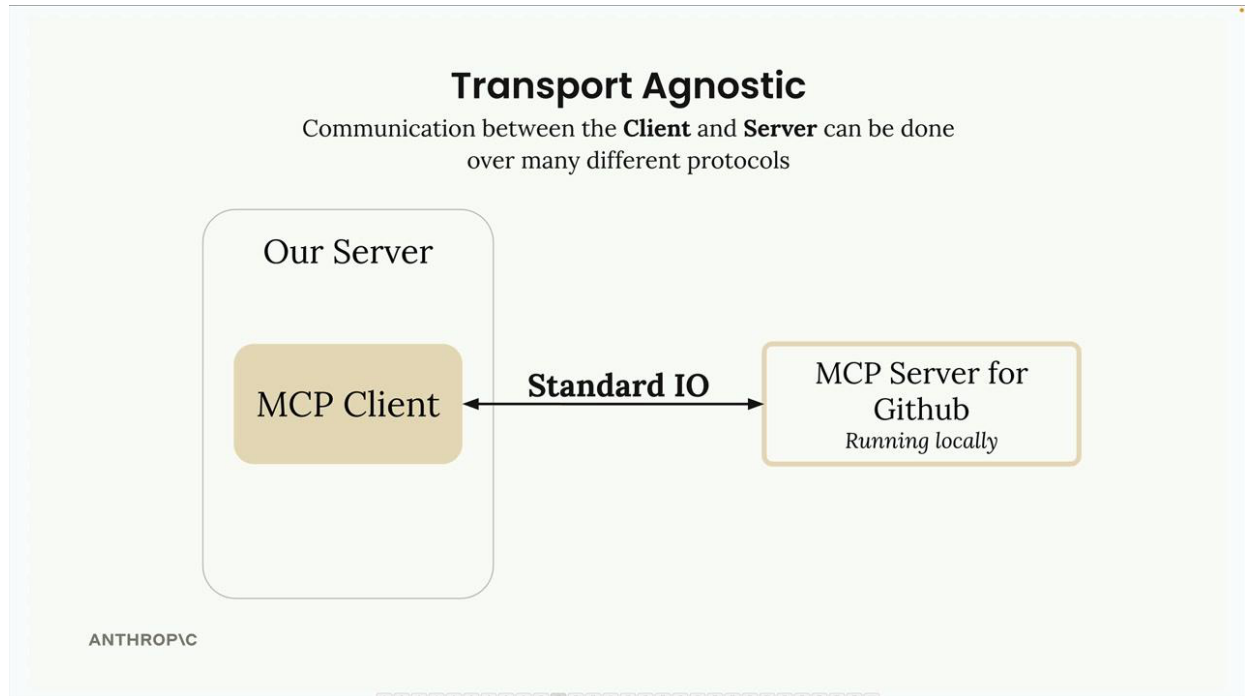
Хіба MCP - це не просто tool use?

Це поширене непорозуміння. MCP servers і tool use є взаємодоповнювальними, але різними concepts. MCP стосується того, хто виконує роботу зі створення та підтримки tools. З MCP хтось інший уже написав tool functions і schemas для вас - вони запаковані всередині MCP server.

Ключова ідея полягає в тому, що MCP servers надають tool schemas і functions, уже визначені для вас, усуваючи потребу самостійно будувати й підтримувати complex integrations.

MCP clients

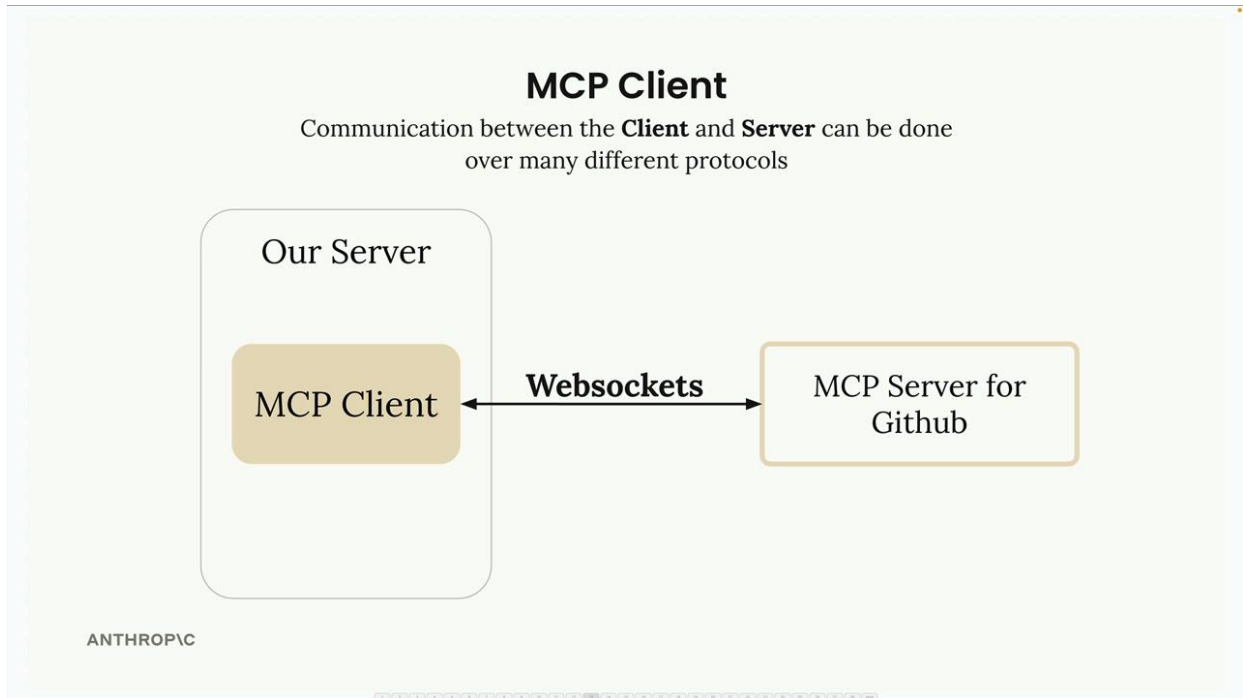
MCP client слугує communication bridge між вашим server і MCP servers. Думайте про нього як про вашу точку доступу до всіх tools, які надає MCP server. Коли вам потрібно використовувати external tools або services, client бере на себе message passing і protocol details.



Transport Agnostic Communication

Одна з ключових переваг MCP - transport agnostic підхід, тобто client і server можуть спілкуватися різними communication methods. Найпоширеніший варіант запускає MCP client і server на одній машині, де вони обмінюються даними через standard input/output.

Але ви не обмежені цим підходом. MCP clients і servers також можуть підключатися через:

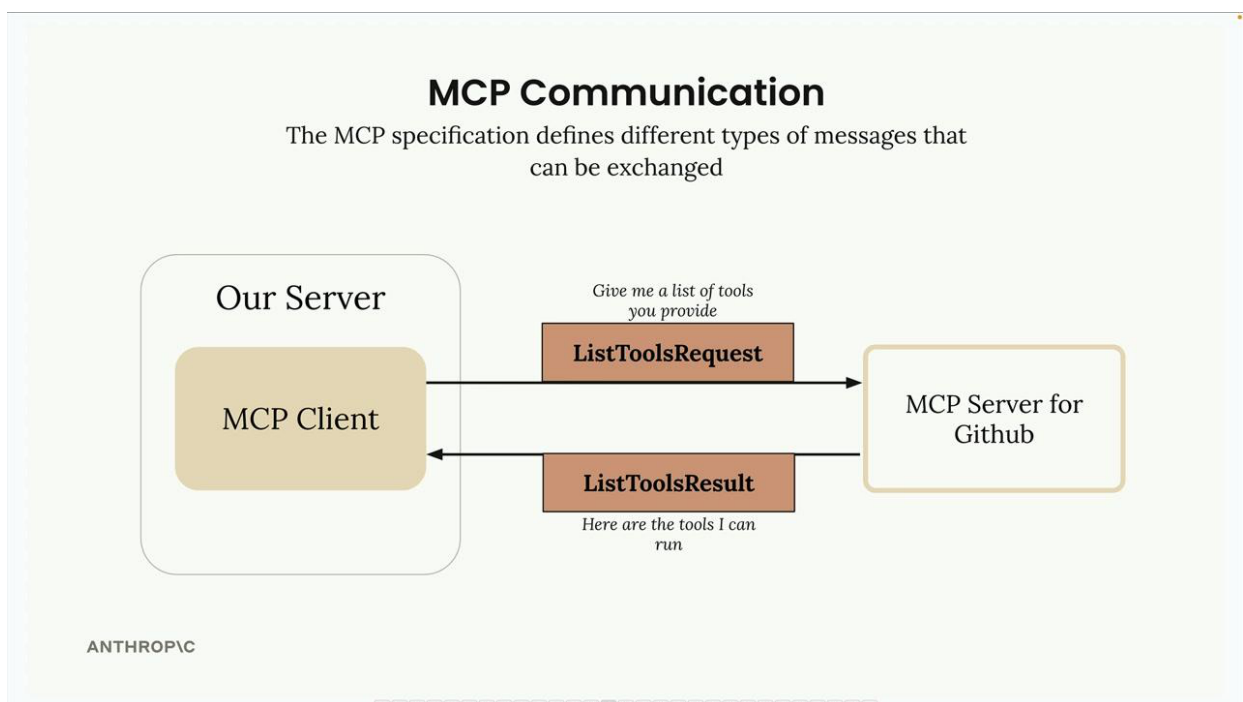
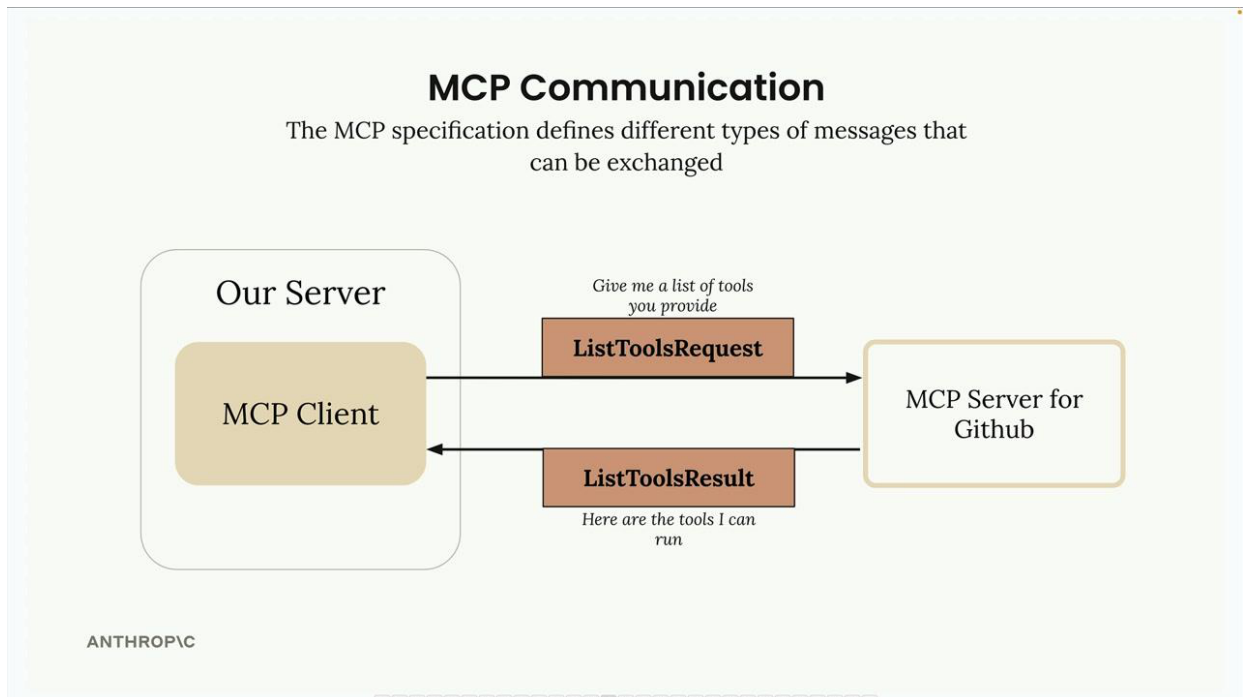


- HTTP
- WebSockets
- Різні інші network protocols

Message Types

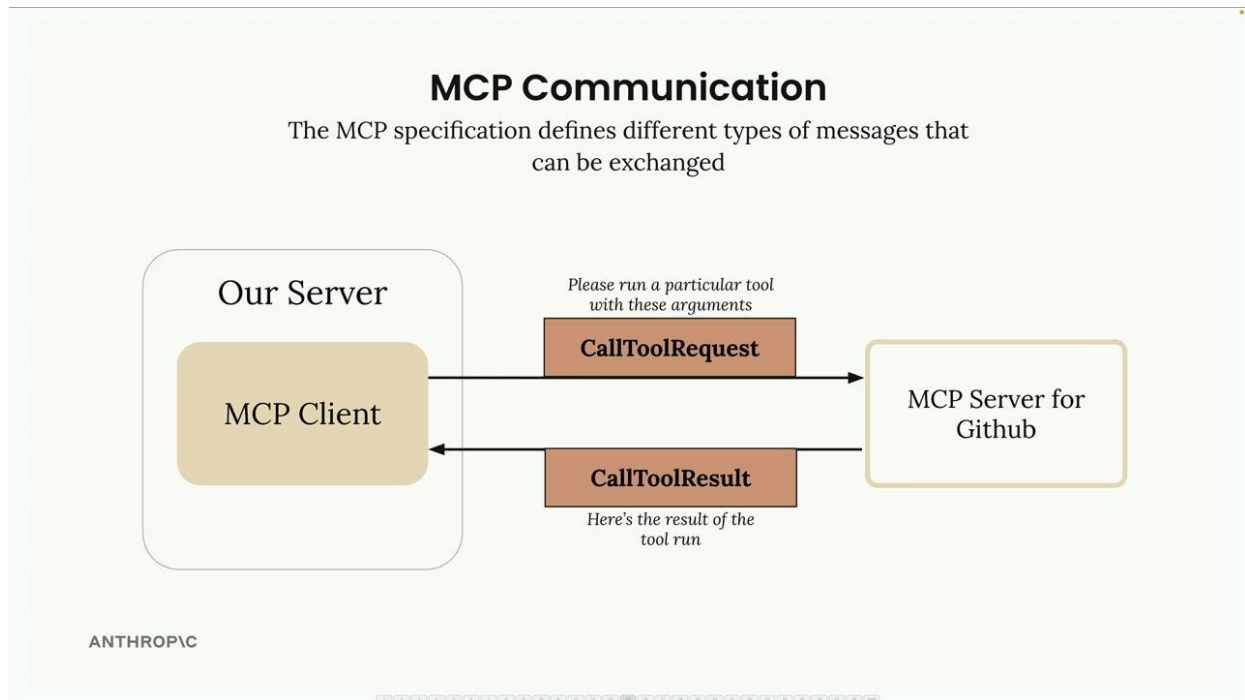
Після підключення client і server обмінюються specific message types, визначеними у MCP specification. Основні message types, з якими ви працюватимете:

ListToolsRequest/ListToolsResult: client запитує у server: "what tools do you provide?" і отримує назад список available tools.



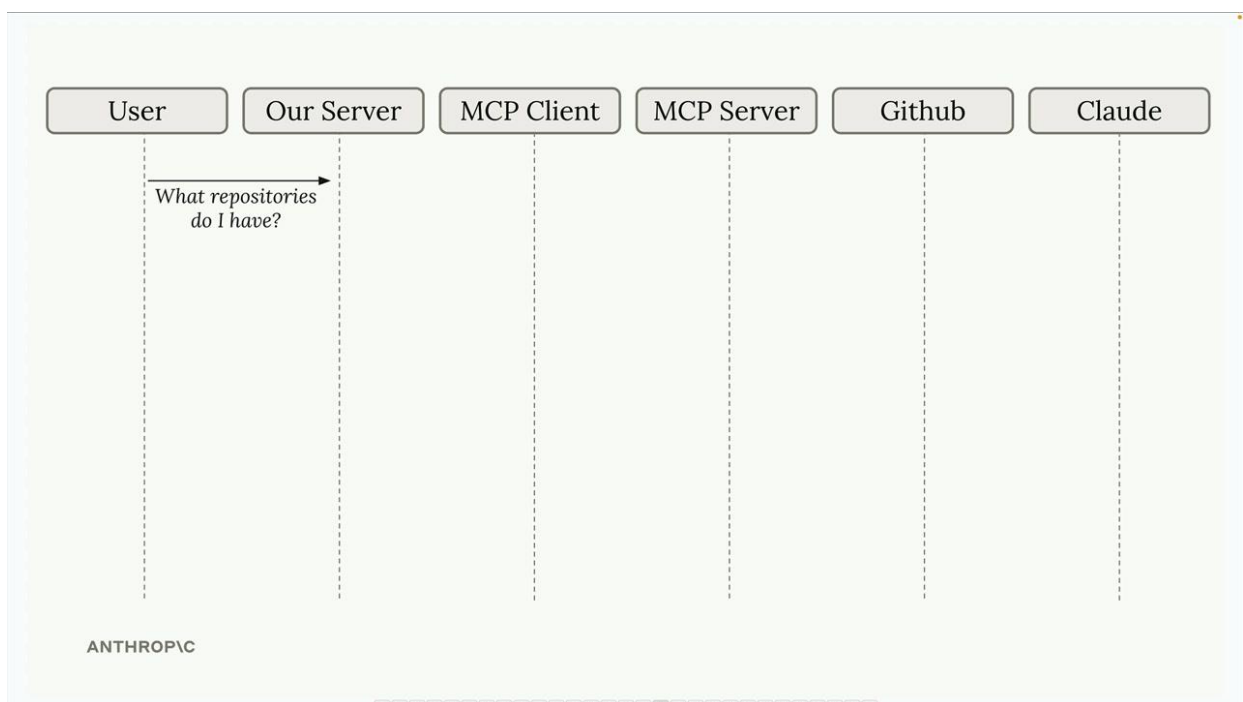
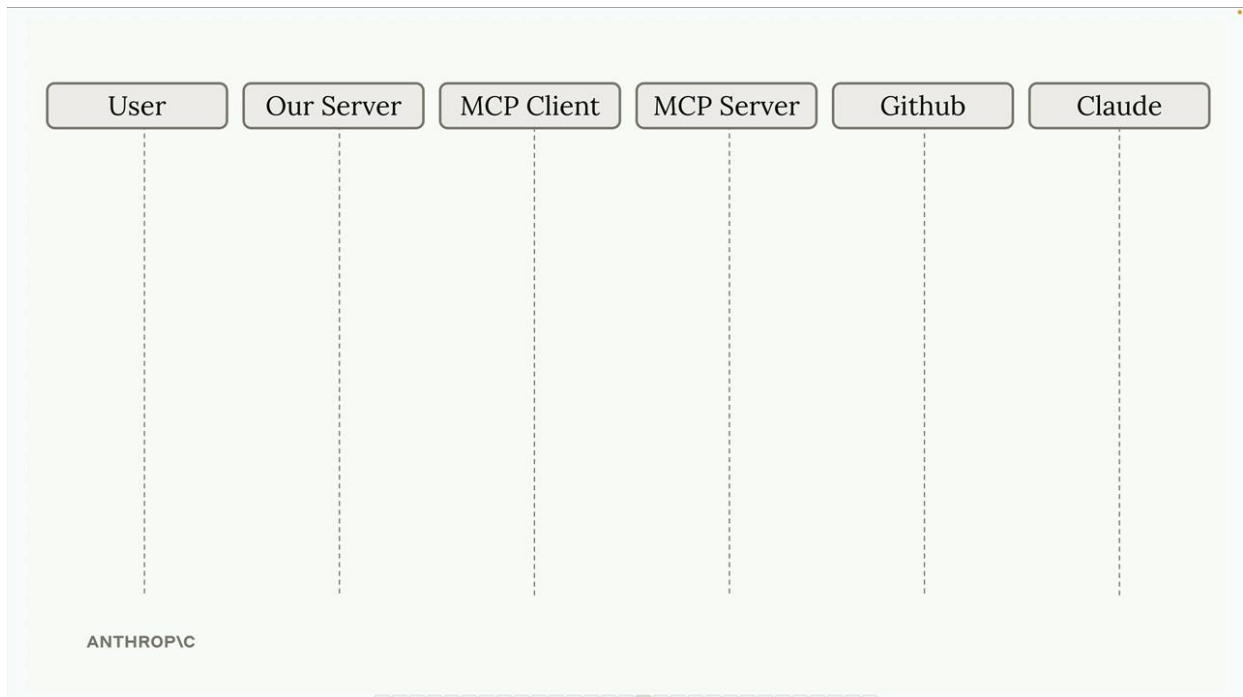
CallToolRequest/CallToolResult: client просить server запустити specific tool з певними arguments, а потім отримує results.

Complete Flow Example



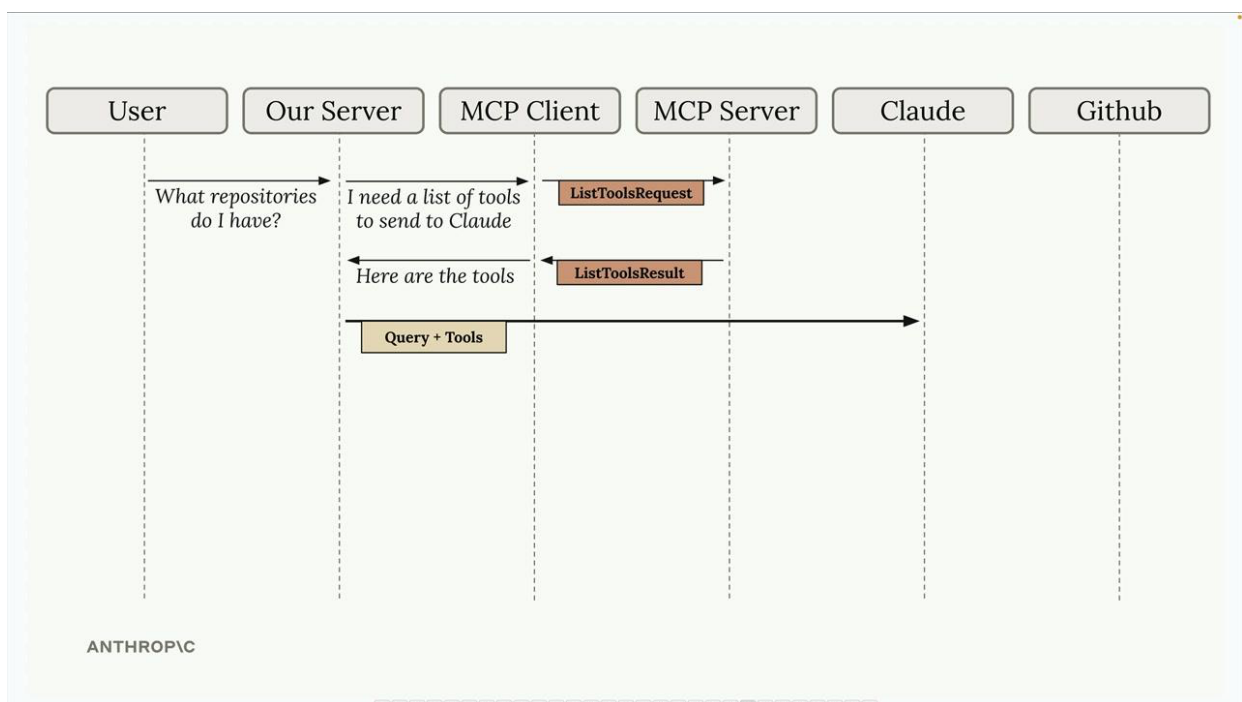
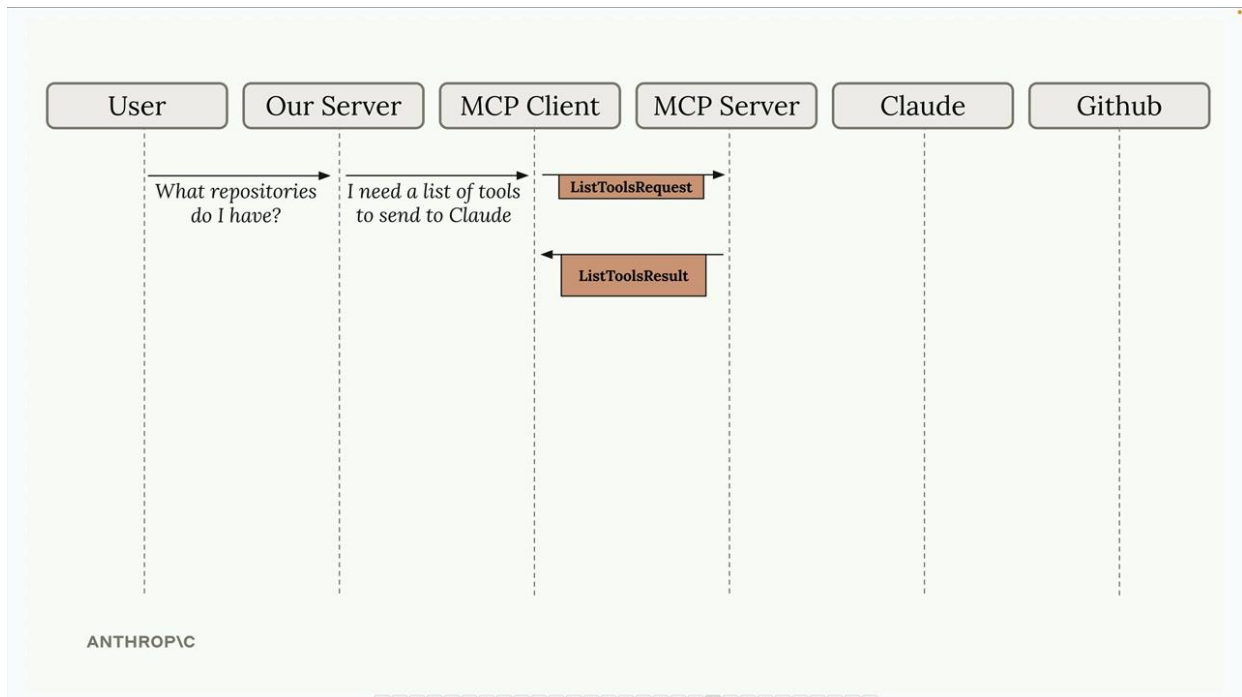
Ось як усі частини працюють разом у реальному сценарії. Припустімо, user питає: "What repositories do I have?" - ось повний communication flow:

Процес починається, коли user надсилає query на ваш server. Ваш server розуміє, що перед виконанням request потрібно надати Claude список available tools.



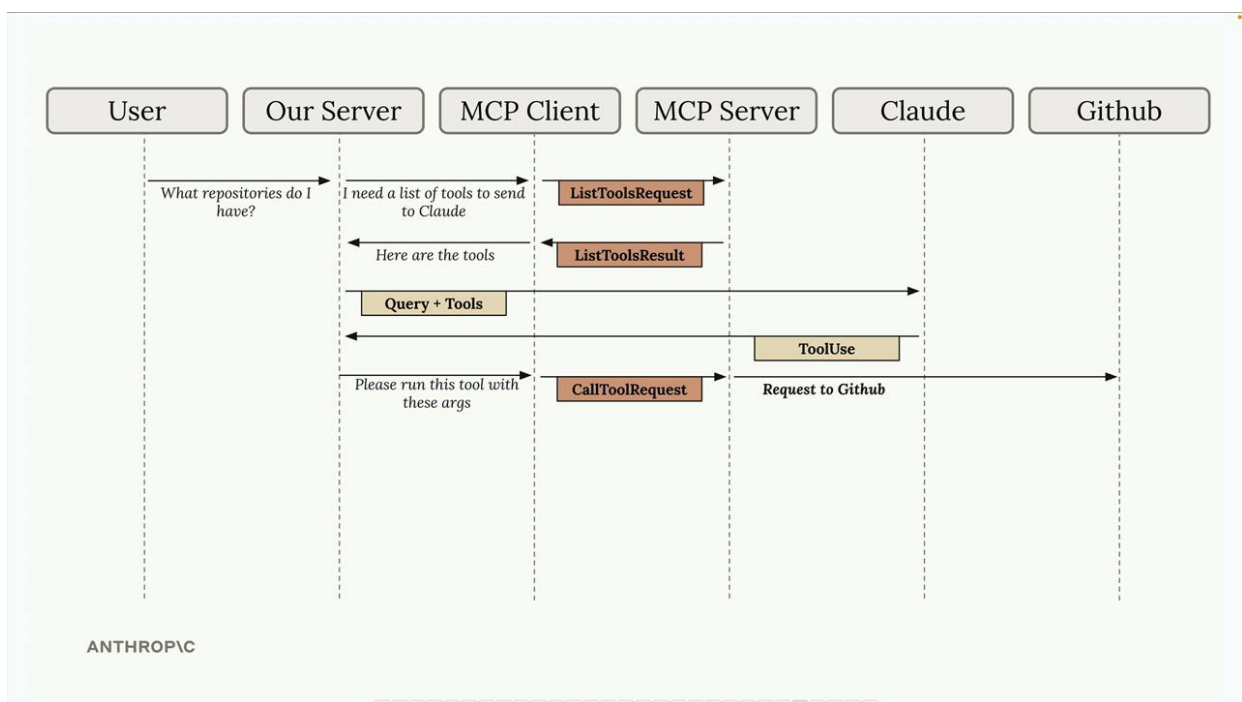
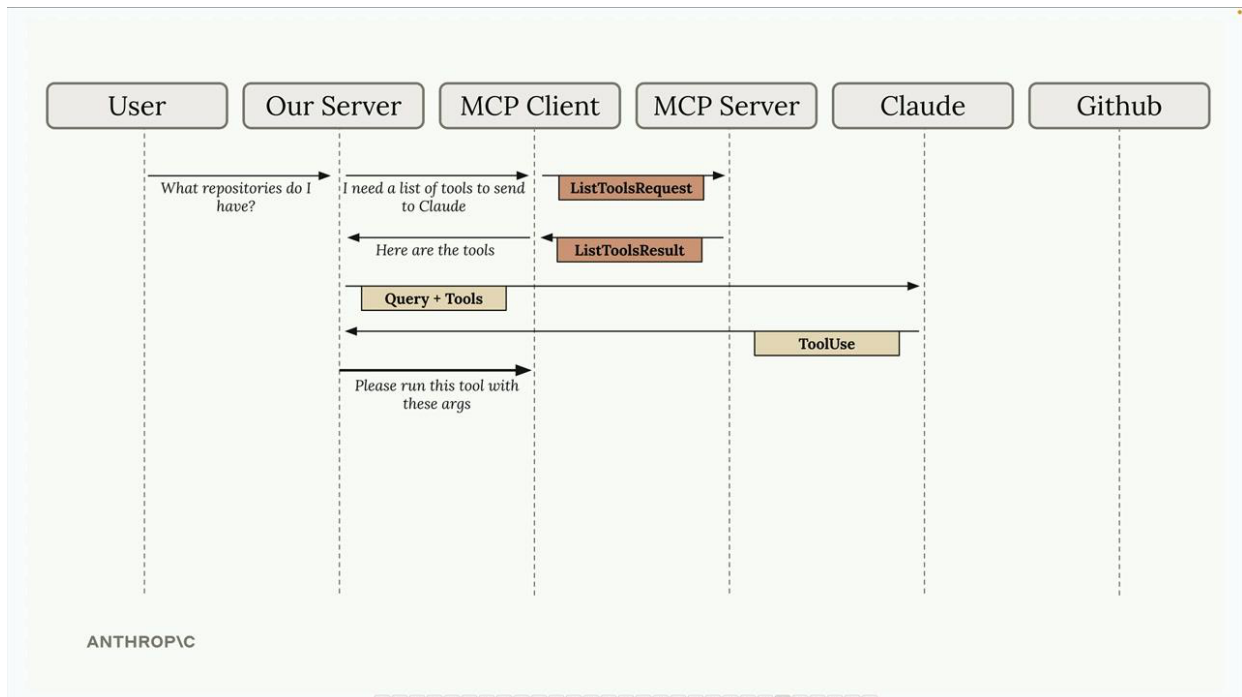
Ваш server просить MCP client отримати tools; той надсилає ListToolsRequest до MCP server і отримує назад ListToolsResult.

Тепер ваш server має все потрібне, щоб зробити initial request до Claude - і питання user, і available tools.



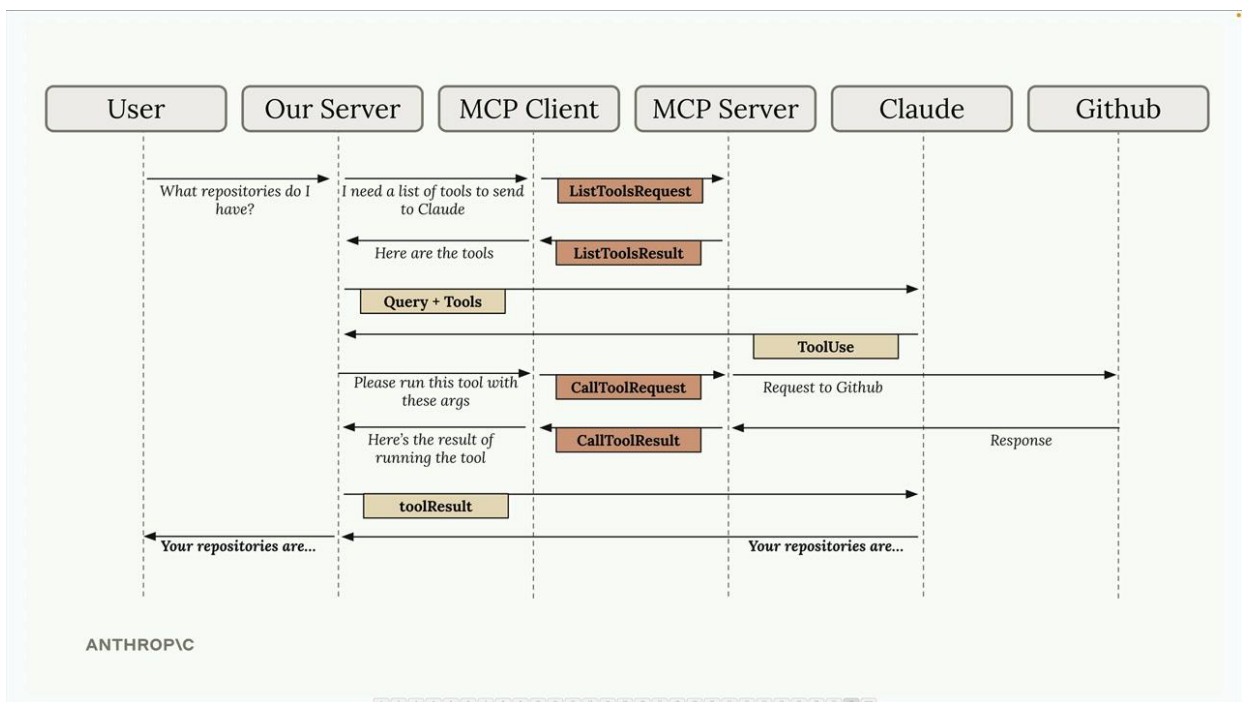
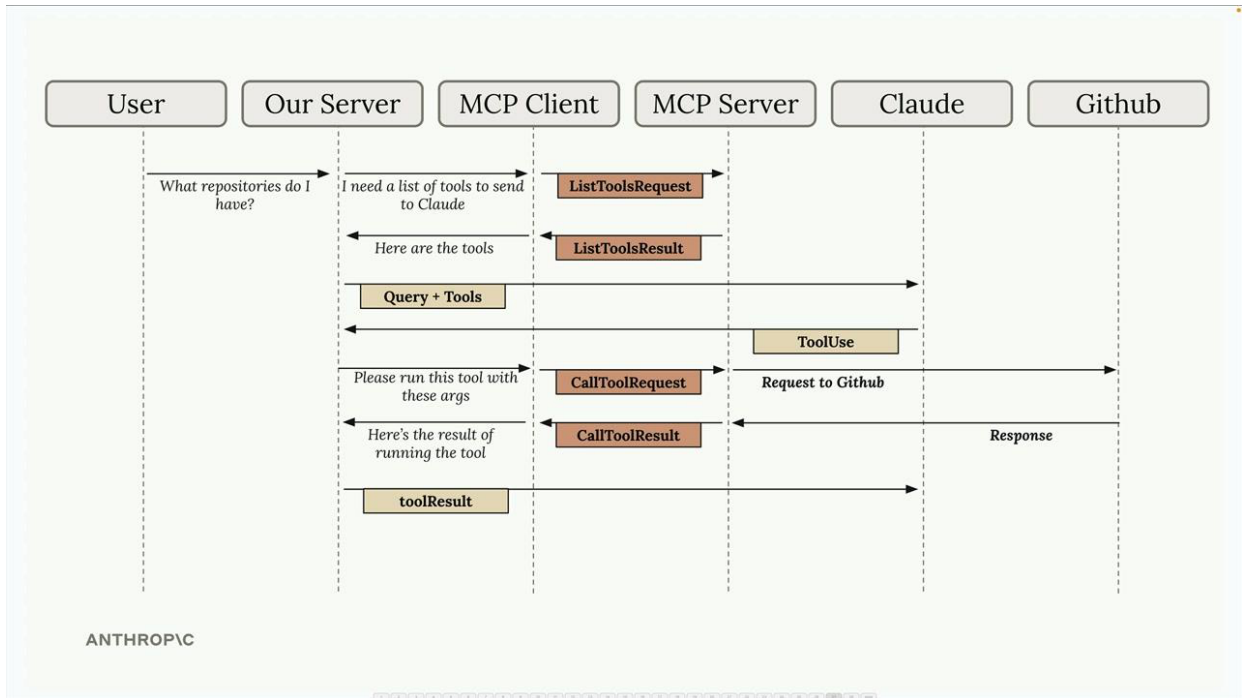
Claude переглядає tools і вирішує, що для відповіді на питання потрібно викликати один із них. Він відповідає tool use request.

Ваш server просить MCP client виконати tool, який запросив Claude. MCP client надсилає CallToolRequest до MCP server, а той виконує фактичний request до Github.



GitHub повертає repository data, які проходять назад через MCP server як CallToolResult, потім до MCP client і нарешті до вашого server.

Ваш server надсилає tool results назад до Claude у follow-up message. Тепер Claude має всю інформацію, потрібну для формування повної відповіді.



Нарешті Claude відповідає formatted answer, яку ваш server передає user.

Так, цей flow містить багато кроків, але кожен component має чітку responsibility. MCP client приховує складність server communication, дозволяючи вам зосередитися на application logic. Коли ми реалізуємо власні MCP client і server, ви побачите, як кожна частина складається в єдину систему на практиці.

Project setup

Downloads

`cli_project.zip` (opens in new tab)

`cli_project_COMPLETE.zip`

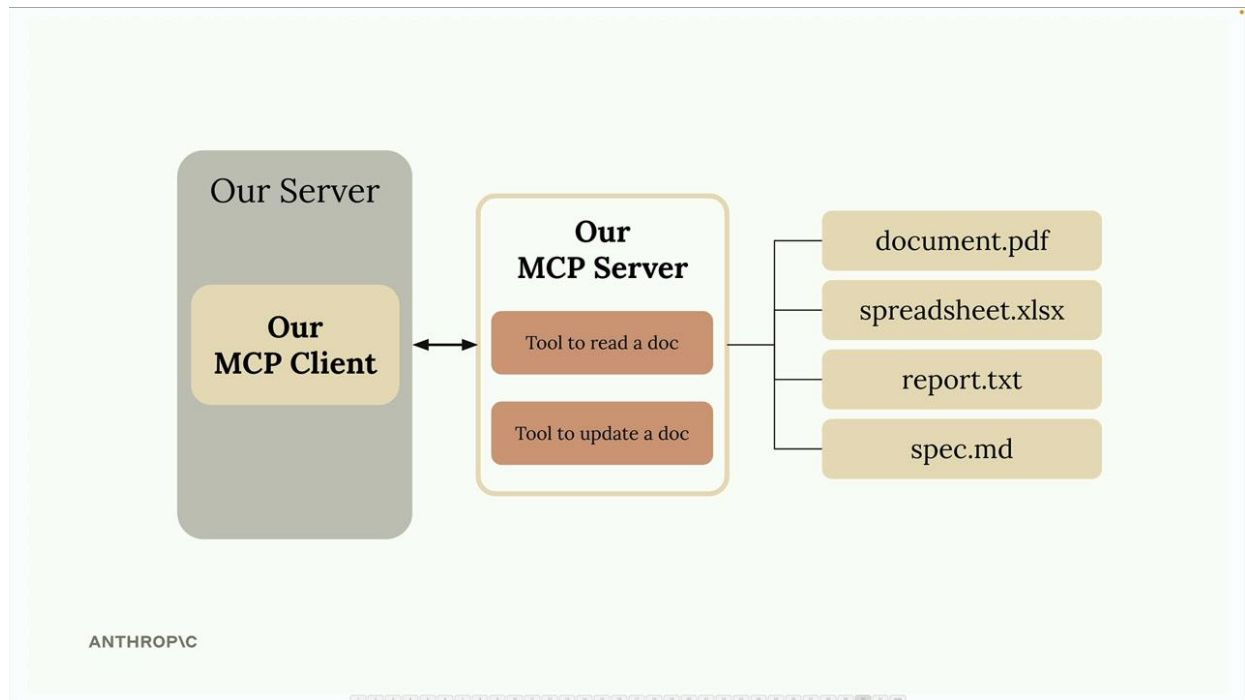
Ми створимо CLI-based chatbot, щоб краще зрозуміти, як MCP clients і servers працюють разом. Цей hands-on project дасть вам практичний досвід з обома сторонами MCP architecture.

Що ми створюємо

Наш chatbot дозволить users взаємодіяти з набором documents через command-line interface. Система складається з двох основних components:

- MCP client, який обробляє user interactions
- Custom MCP server, який керує document operations

Server надаватиме два essential tools: один для читання document contents, а інший для їх оновлення. Для простоти всі documents зберігатимуться in memory - database не потрібна.

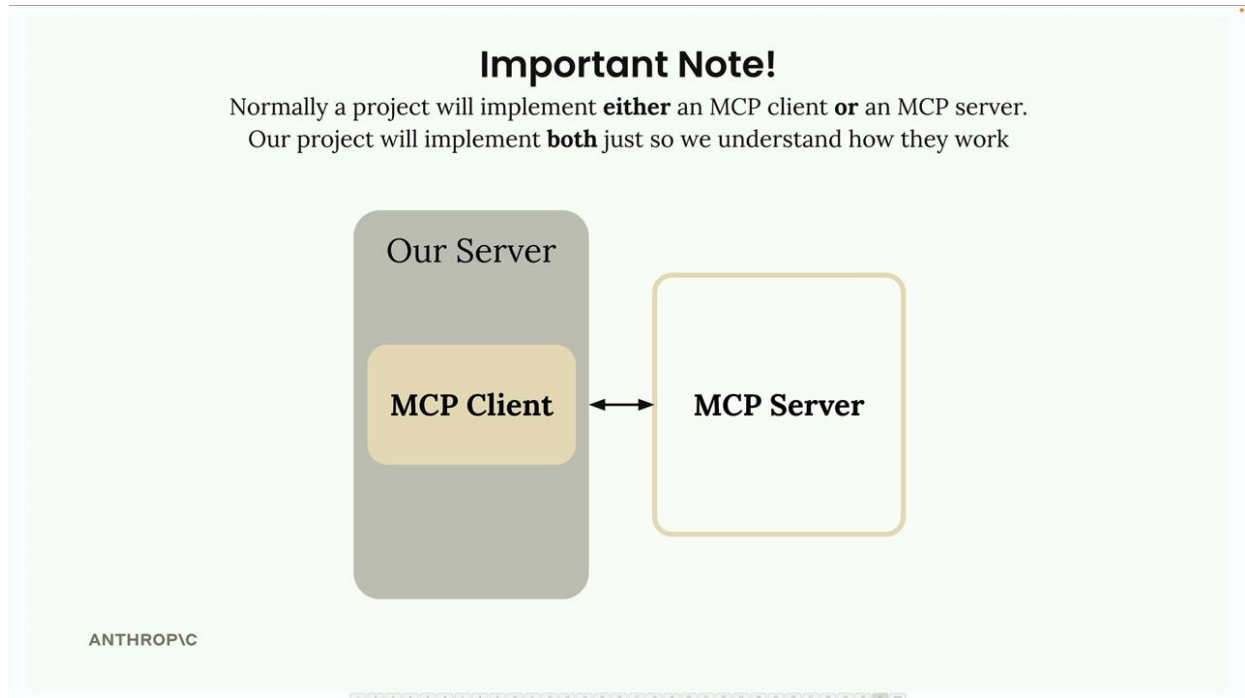


Важлива примітка щодо architecture

У real-world projects зазвичай реалізують або MCP client, або MCP server, але не обидва. Ви можете створити:

- MCP server, щоб відкрити ваш service для інших developers
- MCP client, щоб підключатися до existing MCP servers

У цьому project ми створюємо обидва components виключно з навчальною метою - щоб зрозуміти, як вони спілкуються та працюють разом.



Project Setup

Завантажте файл `cli_project.zip`, прикріплений до цього lesson, і розпакуйте його у бажану development directory. Відкрийте code editor у project folder.

Project містить повний README file з setup instructions. Виконайте такі кроки:

1. Додайте ваш Anthropic API key до `.env` file
2. Встановіть dependencies за допомогою UV (recommended) або pip
3. Запустіть starter application, щоб перевірити, що все працює

Running the Application

Перейдіть до вашої project directory у terminal. Ви побачите основні project files, зокрема main.py, mcp_client.py і mcp_server.py.

Щоб запустити application, використайте одну з цих commands:

```
# If using UV (recommended)
uv run main.py
```

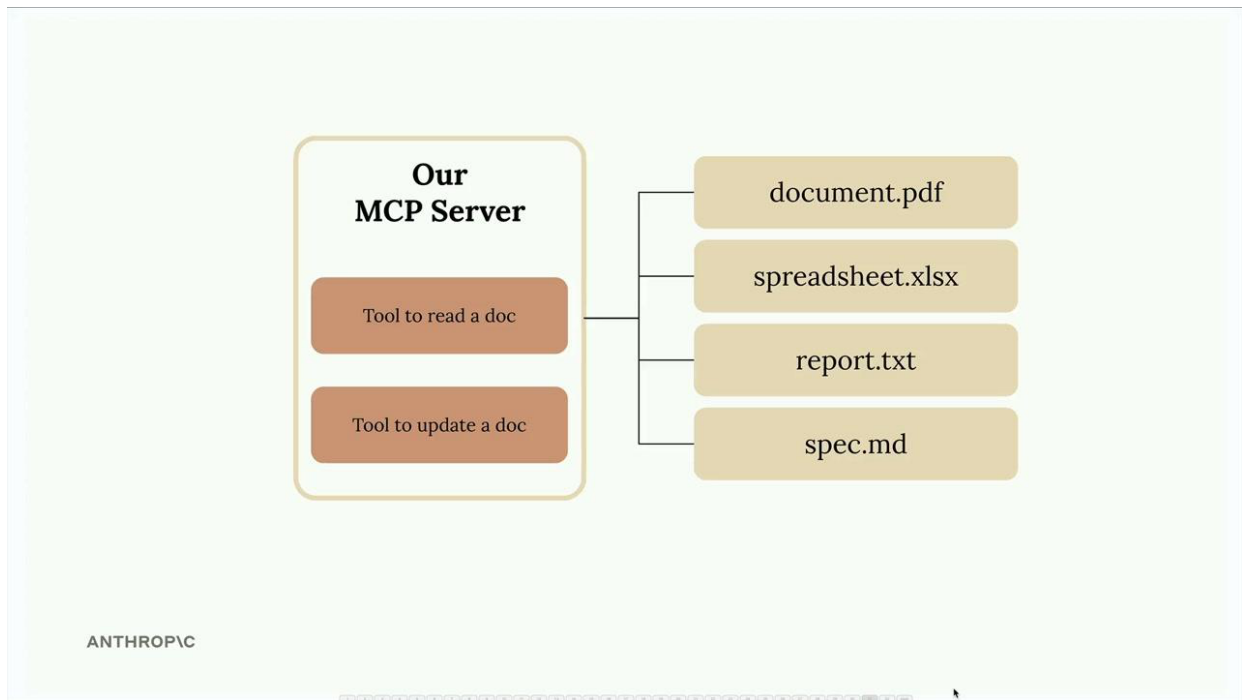
```
# If using standard Python
python main.py
```

Коли application успішно запуститься, ви побачите chat prompt. Протестуйте його простим питанням на кшталт "what's 1+1?" - ви маєте швидко отримати response від Claude.

Коли базовий setup завершено, ми готові почати implementing MCP features і досліджувати, як clients і servers спілкуються через Model Control Protocol.

Defining tools with MCP

Створювати MCP server стає значно простіше, коли ви використовуєте official Python SDK. Замість ручного написання складних JSON schemas для tools, SDK бере цю складність на себе за допомогою decorators і type hints.



У цьому example ми створюємо MCP server, який керує documents, stored in memory. Server надаватиме два essential tools: один для читання document contents, а інший для оновлення через find-and-replace operations.

Setting Up the MCP Server

Python MCP SDK робить створення server надзвичайно простим. Ви можете ініціалізувати повний MCP server одним рядком:

```
from mcp.server.fastmcp import FastMCP

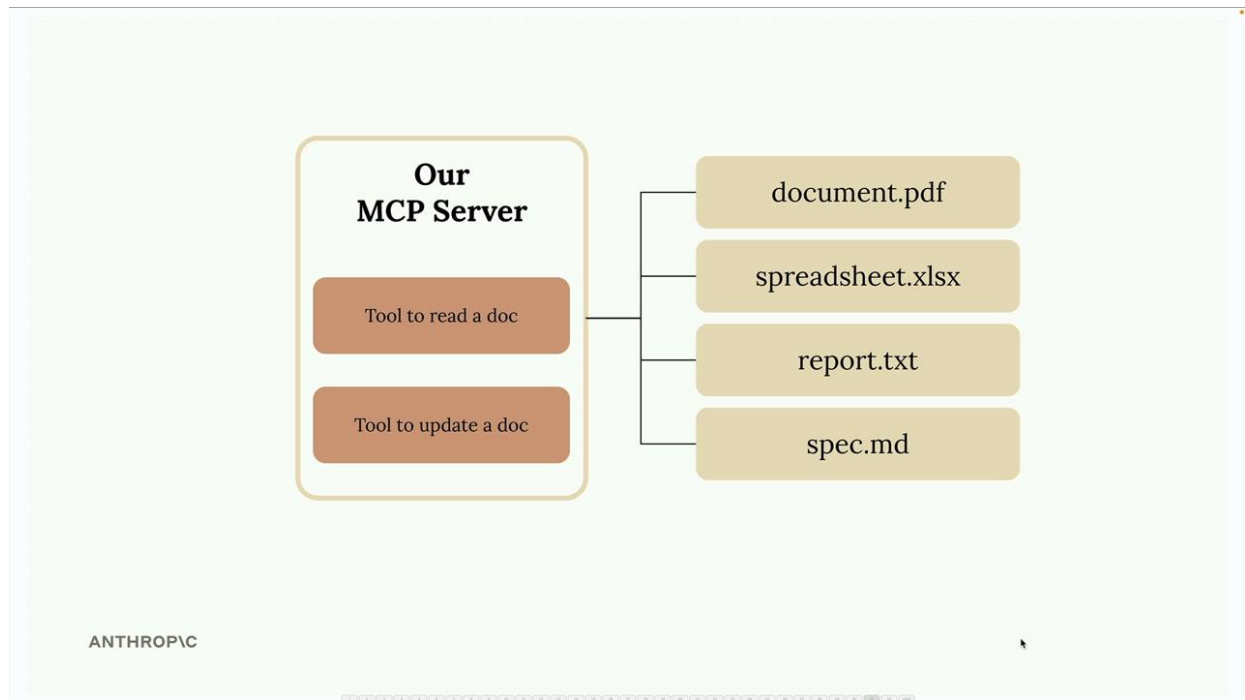
mcp = FastMCP("DocumentMCP", log_level="ERROR")
```

Для цієї implementation documents зберігаються у простому Python dictionary, де keys - це document IDs, а values містять document content:

```
docs = {
    "deposition.md": "This deposition covers the testimony of\nAngela Smith, P.E.",
    "report.pdf": "The report details the state of a 20m\ncondenser tower.",
    "financials.docx": "These financials outline the project's\nbudget and expenditure",
    "outlook.pdf": "This document presents the projected\nfuture performance of the",
    "plan.md": "The plan outlines the steps for the project's\nimplementation.",
    "spec.txt": "These specifications define the technical\nrequirements for the equipment"
}
```

Tool Definition with Decorators

SDK перетворює створення tools із багатослівного процесу на чистий і читабельний code. Замість написання довгих JSON schemas ви використовуєте Python decorators і type hints.



Creating the Document Reader Tool

Перший tool дозволяє Claude прочитати будь-який document за його ID. Ось complete implementation:

```

@mcp.tool(
    name="read_doc_contents",
    description="Read the contents of a document and return it as a string."
)
def read_document(
    doc_id: str = Field(description="Id of the document to read")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
  
```

```
return docs[doc_id]
```

Decorator `@mcp.tool` автоматично генерує JSON schema, потрібну Claude. Class `Field` з Pydantic надає `parameter descriptions`, які допомагають Claude зрозуміти, що очікує кожен `argument`.

Building the Document Editor Tool

Другий tool виконує прості `find-and-replace operations` у documents:

```
@mcp.tool(
    name="edit_document",
    description="Edit a document by replacing a string in the
documents content with a new string."
)
def edit_document(
    doc_id: str = Field(description="Id of the document that
will be edited"),
    old_str: str = Field(description="The text to replace.
Must match exactly, including whitespace."),
    new_str: str = Field(description="The new text to insert
in place of the old text.")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")

    docs[doc_id] = docs[doc_id].replace(old_str, new_str)
```

Цей tool приймає три `parameters`: `document ID`, `text to find` і `replacement text`.

Implementation використовує вбудований Python string method `replace()` для простоти.

Error Handling

Обидва tools містять basic error handling для випадків, коли Claude запитує document, якого не існує. Якщо передано invalid document ID, tools піднімають ValueError з описовим message, який Claude може зрозуміти й потенційно використати.

Key Benefits of the SDK Approach

- Automatic JSON schema generation з Python type hints
- Clean, readable code, який легко підтримувати
- Built-in parameter validation через Pydantic
- Менше boilerplate порівняно з manual schema writing
- Type safety та IDE support для development

MCP Python SDK перетворює те, що раніше було складним процесом написання tool definitions, на щось природне для Python developers. Ви зосереджуєтесь на business logic, а SDK бере на себе protocol details.

The server inspector

Під час створення MCP servers потрібен спосіб тестувати functionality без підключення до повного application. Python MCP SDK містить built-in browser-based inspector, який дозволяє debug і test ваш server у real time.

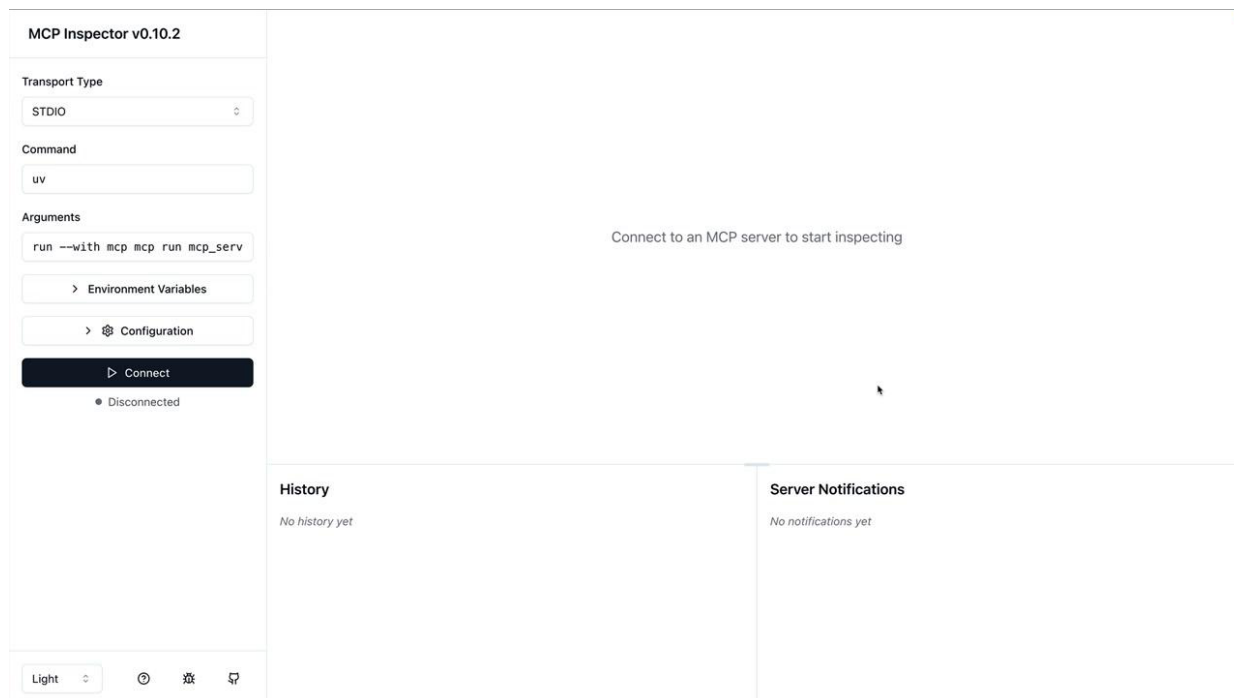
Starting the Inspector

Спершу переконайтеся, що ваше Python environment activated (перевірте README вашого project для точної command). Потім запустіть inspector так:

```
mcp dev mcp_server.py
```

Це запускає development server на port 6277 і дає local URL для відкриття у browser. Inspector interface завантажиться й покаже MCP Inspector dashboard.

Important Note About the Interface

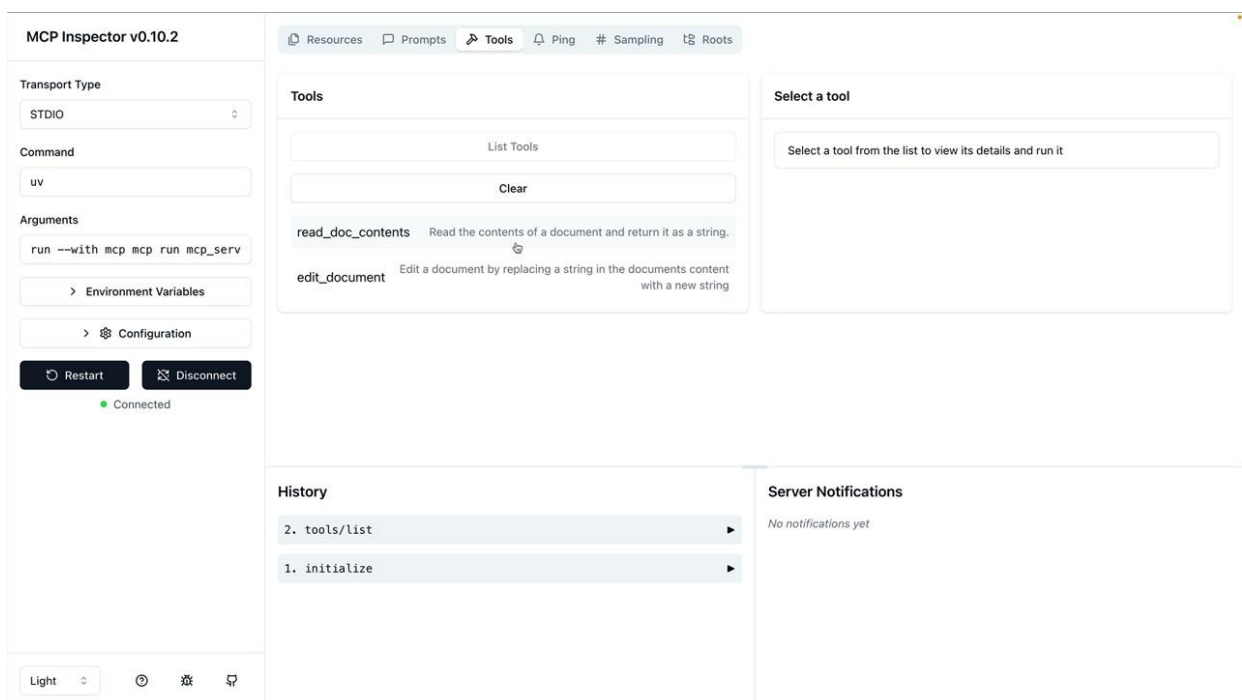
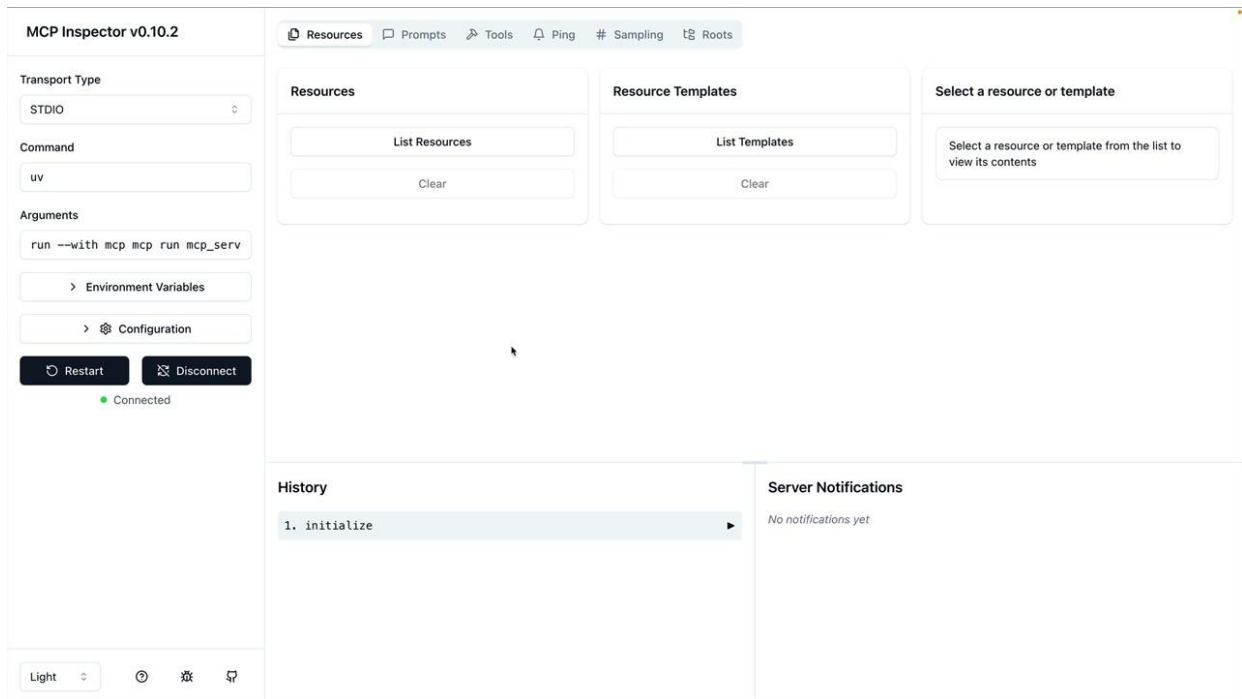


MCP inspector активно розробляється, тому interface, який ви бачите, може відрізнитися від актуальних screenshots. Однак core functionality для testing tools, resources і prompts має залишатися схожою.

Connecting and Testing Tools

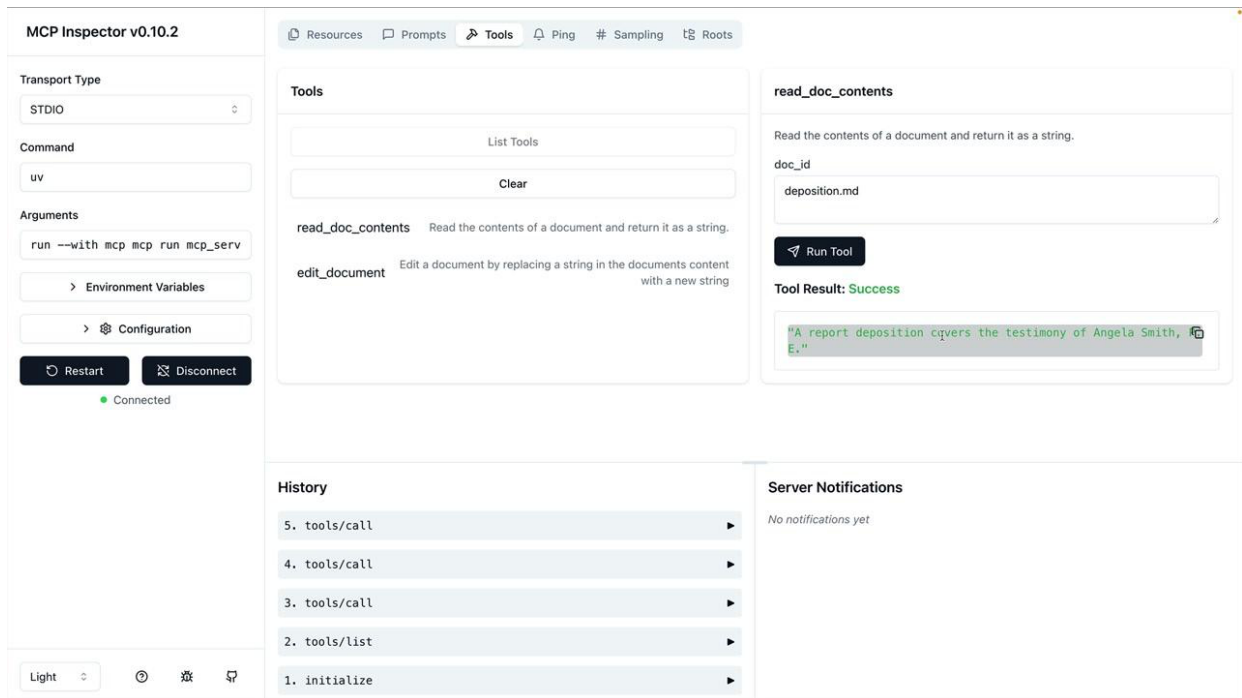
Натисніть кнопку "Connect" зліва, щоб запустити ваш MCP server. Після підключення ви побачите navigation bar із sections для Resources, Prompts, Tools та інших features.

Щоб протестувати tools:



- Перейдіть до section Tools
- Натисніть "List Tools", щоб побачити всі available tools
- Виберіть tool, щоб відкрити його testing interface
- Заповніть required parameters
- Натисніть "Run Tool", щоб виконати й побачити results

Testing Document Operations



Наприклад, щоб протестувати document reading tool, ви вводите document ID (наприклад, "deposition.md") і запускаєте tool. Inspector показує result, включно з returned content або success messages.

Ви можете chain operations, щоб перевірити functionality. Наприклад, після редагування document через replacing text можна одразу знову запустити read tool, щоб підтвердити, що changes були applied correctly.

Development Workflow

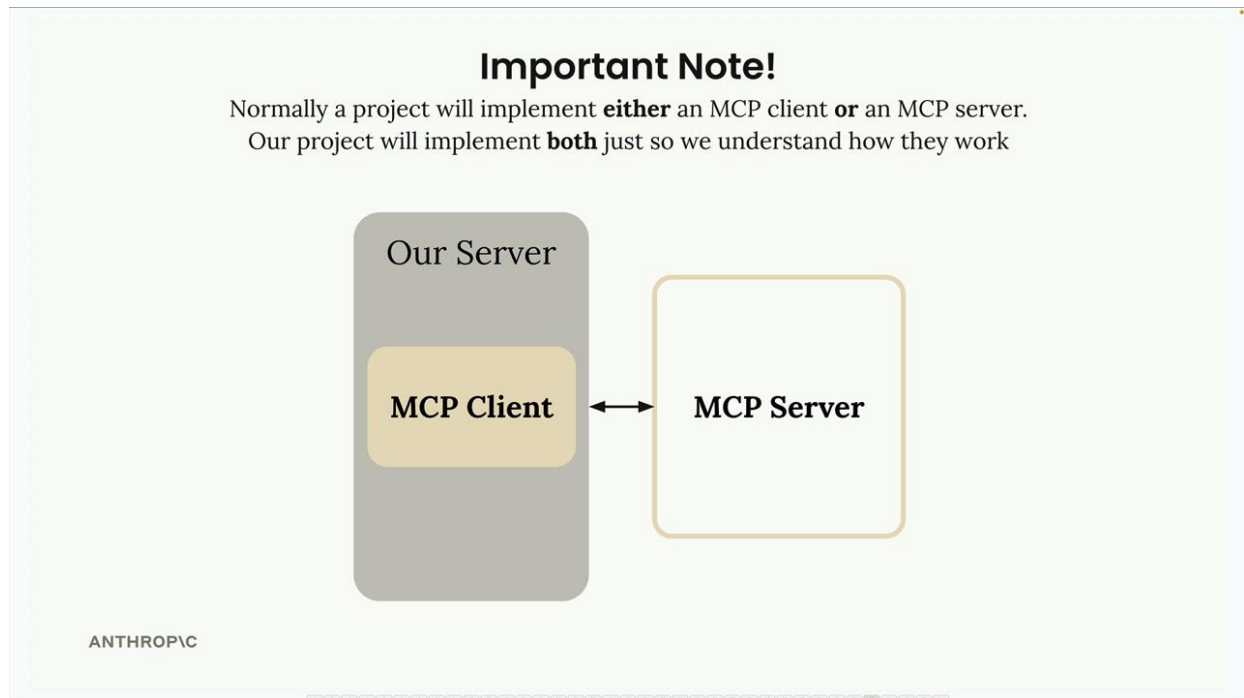
Inspector створює efficient development loop:

- Вносите changes у code вашого MCP server

- Тестуєте individual tools через inspector
- Перевіряєте results без потреби налаштовувати full application
- Debug issues in isolation

Цей tool стає essential, коли ви створюєте складніші MCP servers. Він усуває потребу підключати server до Claude або іншого application лише для тестування базової functionality, роблячи development набагато швидшим і focused.

Implementing a client



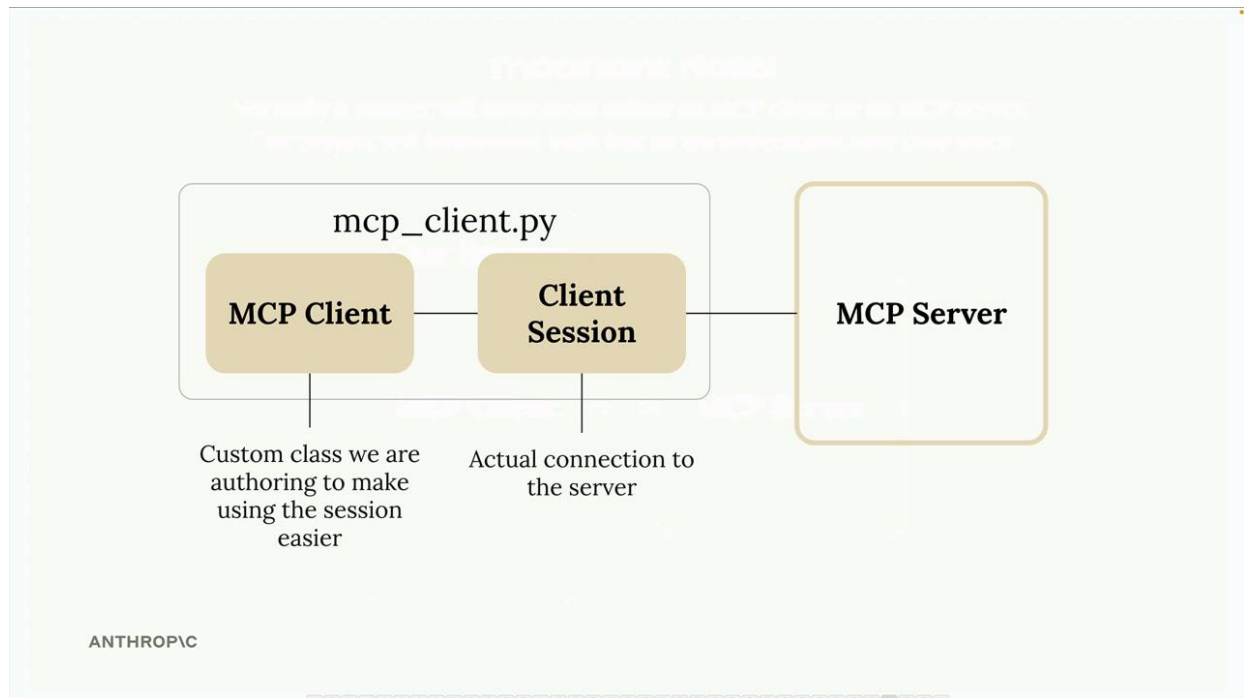
Тепер, коли наш MCP server працює, настав час створити client side. Client - це те, що дозволяє нашому application спілкуватися з MCP server і отримувати доступ до його functionality.

Understanding the Client Architecture

У більшості real-world projects ви реалізуєте або MCP client, або MCP server - не обидва. У цьому project ми створюємо обидва лише для того, щоб ви побачили, як вони працюють разом.

MCP client складається з двох основних components:

- MCP Client - custom class, який ми створюємо, щоб спростити використання session

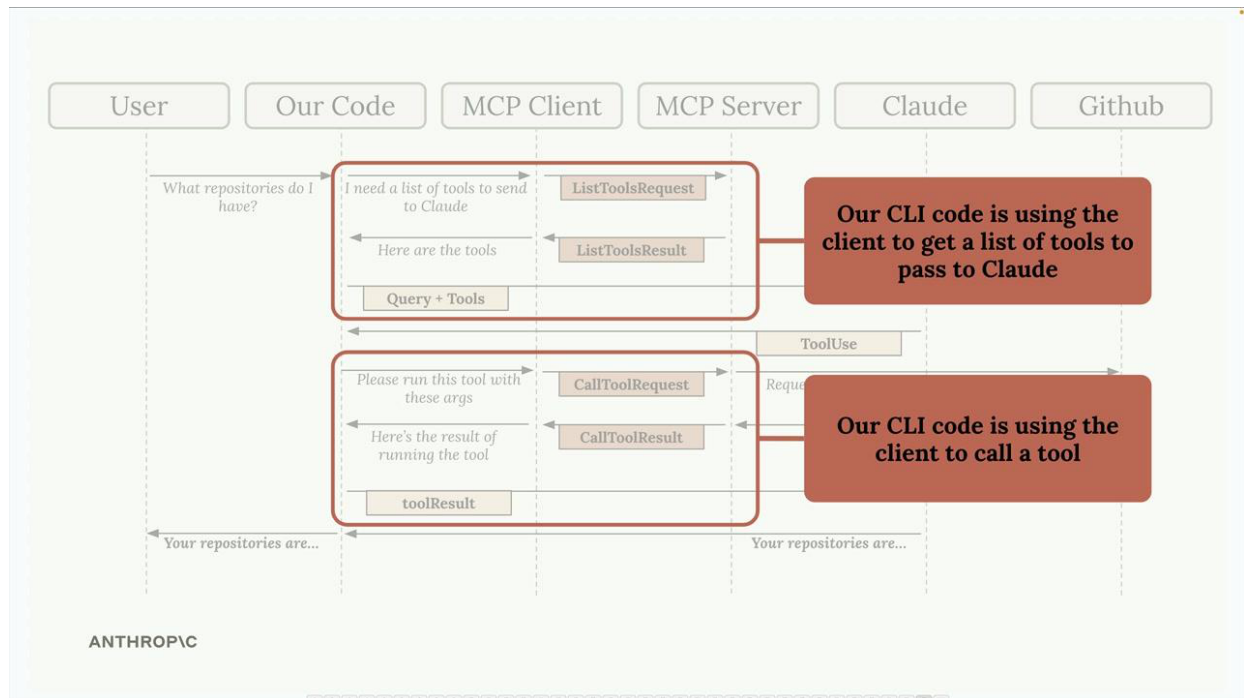


- Client Session - фактичне connection до server (частина MCP Python SDK)

Client session потребує правильного resource cleanup, коли ми з нею завершили роботу. Саме тому ми wrap її в наш custom MCP Client class - щоб автоматично обробляти весь cleanup.

How the Client Fits Into Our Application

Пам'ятаєте наш application flow? Наш CLI code має робити дві основні речі з MCP server:



1. Отримати список available tools для надсилання Claude

2. Виконувати tools, коли Claude їх запитує

MCP client надає ці capabilities через прості method calls, які може використовувати наш application code.

Implementing the Core Methods

Нам потрібно реалізувати два ключові methods у нашому client: `list_tools()` і `call_tool()`.

List Tools Method

Цей method отримує всі available tools із server:

```

async def list_tools(self) -> list[types.Tool]:
    result = await self.session().list_tools()
    
```

```
        return result.tools
```

Усе просто - ми звертаємося до нашої session (connection до server), викликаємо built-in function list_tools() і повертаємо tools з result.

Call Tool Method

Цей method виконує specific tool на server:

```
async def call_tool(
    self, tool_name: str, tool_input: dict
) -> types.CallToolResult | None:
    return await self.session().call_tool(tool_name,
    tool_input)
```

Ми передаємо tool name і input parameters (надані Claude) до server і повертаємо result.

Testing the Client

Щоб протестувати implementation, можна запустити client напямую. File містить testing harness, який підключається до нашого MCP server і викликає methods:

```
async with MCPClient(
    command="uv", args=["run", "mcp_server.py"]
) as client:
    result = await client.list_tools()
    print(result)
```

Коли ми запускаємо цей test, маємо побачити надруковані tool definitions, зокрема tools `read_doc_contents` і `edit_document`, які ми створили раніше.

Putting It All Together

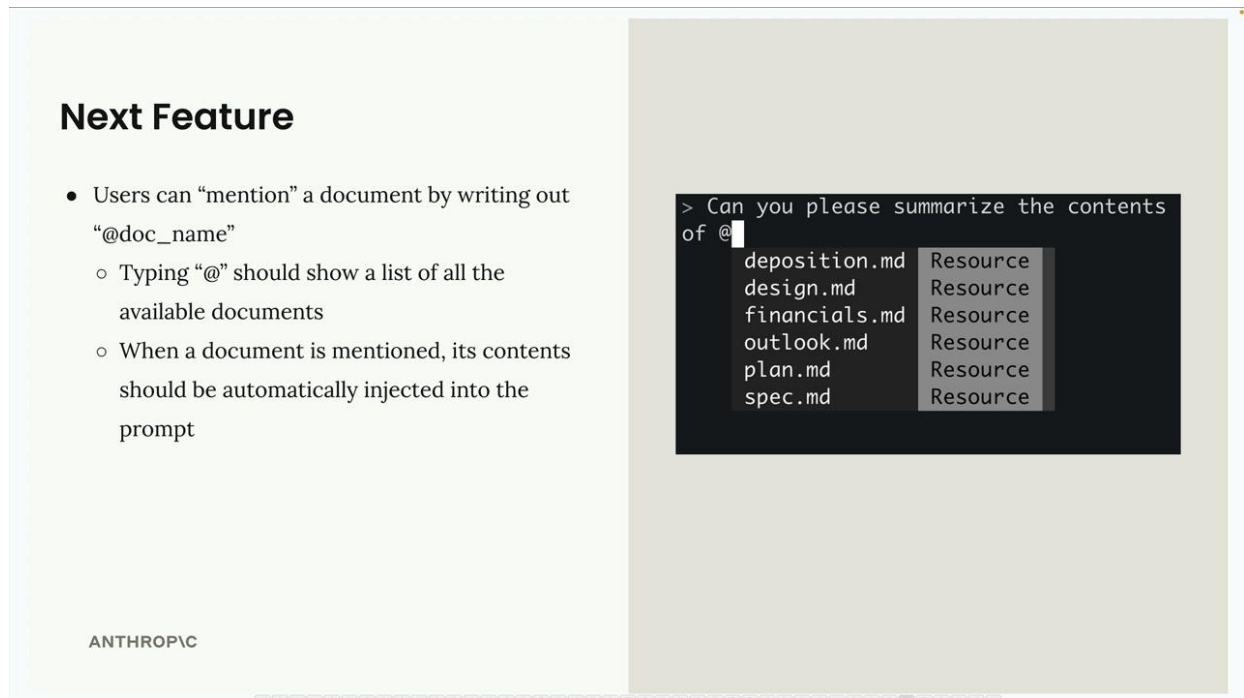
Тепер, коли наш client може list tools і call them, ми можемо протестувати complete flow. Коли ми запускаємо main application і питаємо Claude про document:

1. Наш code використовує client, щоб отримати available tools
2. Ці tools надсилаються Claude разом із question user
3. Claude вирішує використати tool `read_doc_contents`
4. Наш code використовує client, щоб виконати цей tool
5. Result надсилається назад Claude, який потім відповідає user

Наприклад, питання "What is the contents of the report.pdf document?" змусить Claude використати наш document reading tool, і ми отримаємо інформацію про 20m condenser tower document, який налаштували у server.

Client діє як bridge між application logic і MCP server, спрощуючи доступ до server functionality без необхідності думати про underlying connection details.

Defining resources

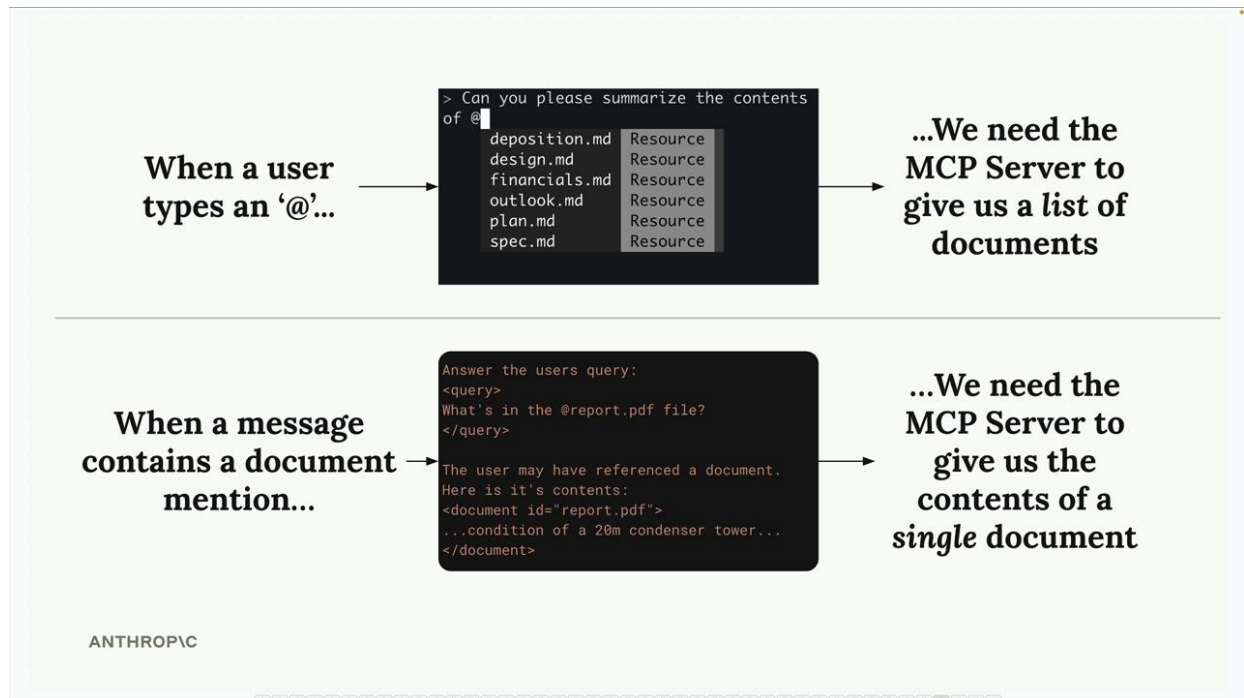


Resources у MCP servers дозволяють відкривати data для clients, подібно до GET request handlers у типовому HTTP server. Вони ідеально підходять для scenarios, де потрібно fetch information, а не виконувати actions.

Understanding Resources Through an Example

Уявімо, що ви хочете створити document mention feature, де users можуть вводити @document_name, щоб reference files. Для цього потрібні дві operations:

- Отримати список усіх available documents (для autocomplete)
- Отримати contents specific document (коли його mentioned)



Коли user вводить @, потрібно показати available documents. Коли він надсилає message з mention, ви автоматично вставляєте content цього document у prompt, що надсилається Claude.

How Resources Work

Resources працюють за request-response pattern. Ваш client надсилає ReadResourceRequest з URI, а MCP server відповідає data. URI працює як address для resource, до якого ви хочете отримати доступ.

Resources

- Allow the MCP Server to expose data to the client
- Similar to GET request handlers in a typical HTTP server
- Can return any type of data - strings, JSON, binary, etc.
 - We set the 'mime_type' to give the client a hint as to what data we are returning
- Two types: direct and templated

ANTHROPIC

MCP Server

Resource

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

Resource +

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

```
@mcp.resource(
    "docs://documents", # URI
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

Direct Resource

URI doesn't contain any params.

ANTHROPIC

```
@mcp.resource(
    "docs://documents/{doc_id}", # URI
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

Templated Resource

URI contains one or more params. The Python SDK parses these and passes them as args to your function.

Types of Resources

Є два types of resources:

- Direct Resources: static URIs, які не змінюються, наприклад docs://documents
- Templated Resources: URIs з parameters, наприклад docs://documents/{doc_id}

Для templated resources Python SDK автоматично parses parameters з URI і передає їх як keyword arguments до вашої function.

Implementing Resources

Resources визначаються за допомогою decorator `@mcp.resource()`. Ось як створити обидва types:

Direct Resource (List Documents)

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs() -> list[str]:
    return list(docs.keys())
```

Templated Resource (Fetch Document)

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str) -> str:
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
```

```
return docs[doc_id]
```

MIME Types

Resources можуть повертати будь-який type of data - strings, JSON, binary тощо. Parameter `mime_type` дає clients підказку про те, який kind of data ви повертаєте:

- `application/json` - structured JSON data
- `text/plain` - plain text content
- Будь-який інший valid MIME type для різних data formats

MCP Python SDK автоматично serializes ваші return values. Вам не потрібно вручну convert to JSON strings.

Testing Resources

Ви можете тестувати resources за допомогою MCP Inspector. Запустіть server так:

```
uv run mcp dev mcp_server.py
```

Потім підключіться до inspector у browser. Ви побачите:

- Resources: показує ваші direct/static resources
- Resource Templates: показує templated resources, які приймають parameters

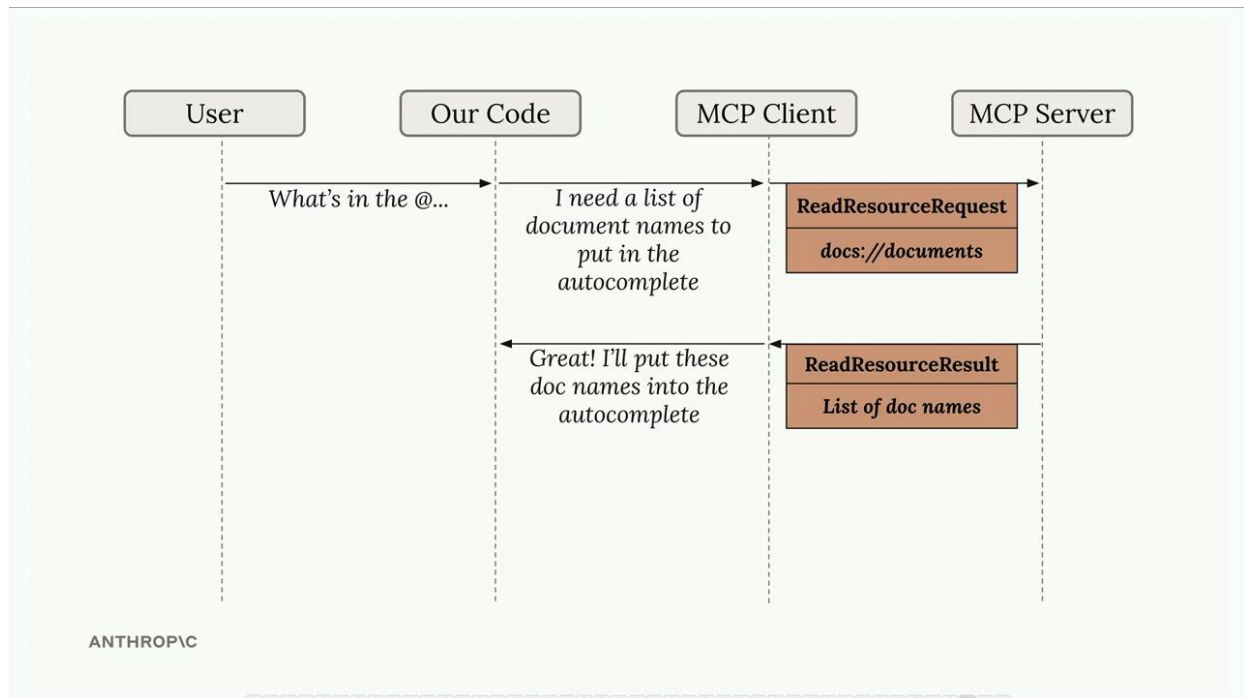
Натисніть будь-який resource, щоб протестувати його і побачити exact response structure, яку отримає ваш client.

Key Points

- Resources expose data, tools perform actions
- Використовуйте direct resources для static data, templated resources - для parameterized queries
- MIME types допомагають clients зрозуміти response format
- SDK handles serialization automatically
- Parameter names у templated URIs стають function arguments

Resources надають чистий спосіб зробити data доступними для MCP clients, enabling features на кшталт document mentions, file browsing або будь-якого scenario, де потрібно fetch information з вашого server.

Accessing resources



Resources у MCP дозволяють вашому server відкривати data, які можна directly included in prompts, замість того щоб вимагати tool calls для доступу до information. Це створює ефективніший спосіб надати context AI models на кшталт Claude.

Understanding Resource Requests

Коли ви визначили resources на своєму MCP server, ваш client потребує способу request і use them. Client діє як bridge між вашим application і MCP server, автоматично обробляючи communication і data parsing.

Flow простий: коли user хоче reference document (наприклад, typing "@report.pdf"), ваш application використовує MCP client, щоб fetch

цей resource із server і включити його contents напяму в prompt, який надсилається Claude.

Implementing Resource Reading

Core functionality потребує function `read_resource` у вашому MCP client. Ця function приймає URI parameter, який визначає, який resource fetch:

```
async def read_resource(self, uri: str) -> Any:
    result = await self.session().read_resource(AnyUrl(uri))
    resource = result.contents[0]
```

Response від MCP server містить contents list. Зазвичай потрібен лише first element, який містить actual resource data разом із metadata, наприклад MIME type.

Handling Different Content Types

Resources можуть повертати різні content types, тому client має parse them appropriately. MIME type підказує, як обробляти data:

```
if isinstance(resource, types.TextResourceContents):
    if resource.mimeType == "application/json":
        return json.loads(resource.text)

    return resource.text
```

Такий approach гарантує, що JSON resources правильно parsed into Python objects, а plain text resources повертаються як strings. MIME type діє як hint для визначення correct parsing strategy.

Required Imports

Щоб усе працювало правильно, у MCP client потрібні такі imports:

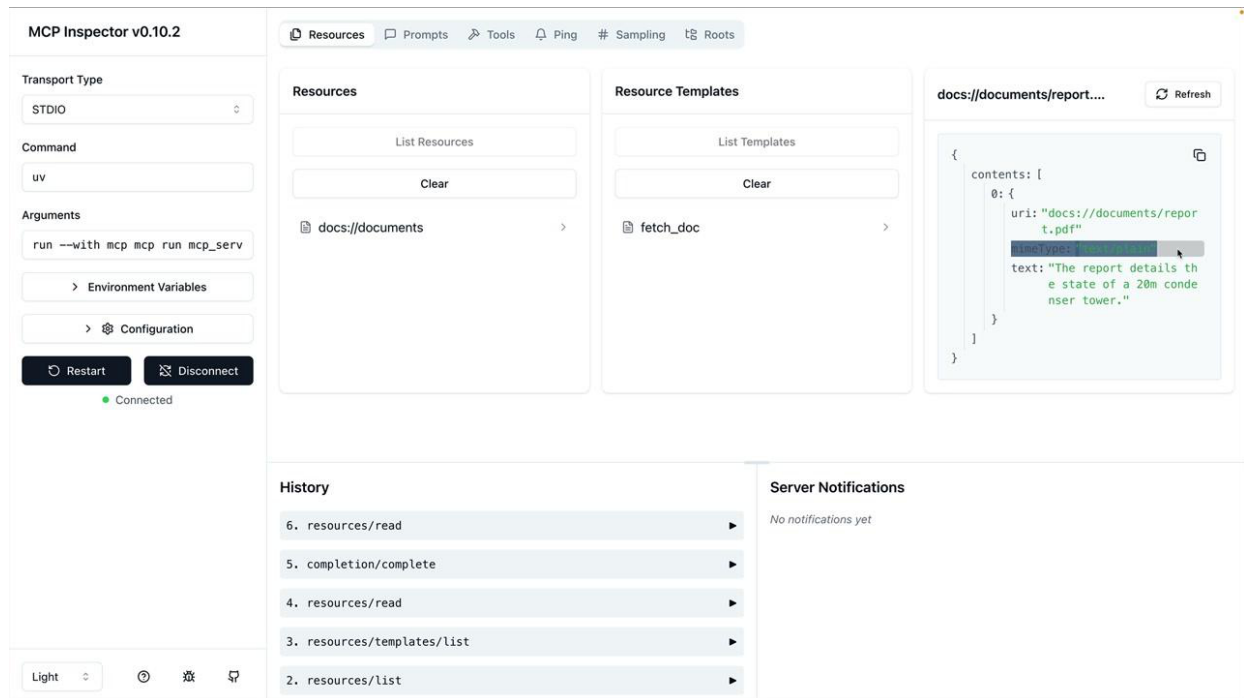
```
import json
from pydantic import AnyUrl
```

Module json обробляє parsing JSON responses, а AnyUrl забезпечує proper type handling для URI parameter.

Testing Resource Access

Після implementation можна тестувати functionality через CLI application. Коли ви вводите щось на кшталт "What's in the @report.pdf document?", system має:

- Показати available resources у autocomplete list
- Дозволити вибрати resource
- Автоматично fetch resource content
- Include that content in the prompt to Claude



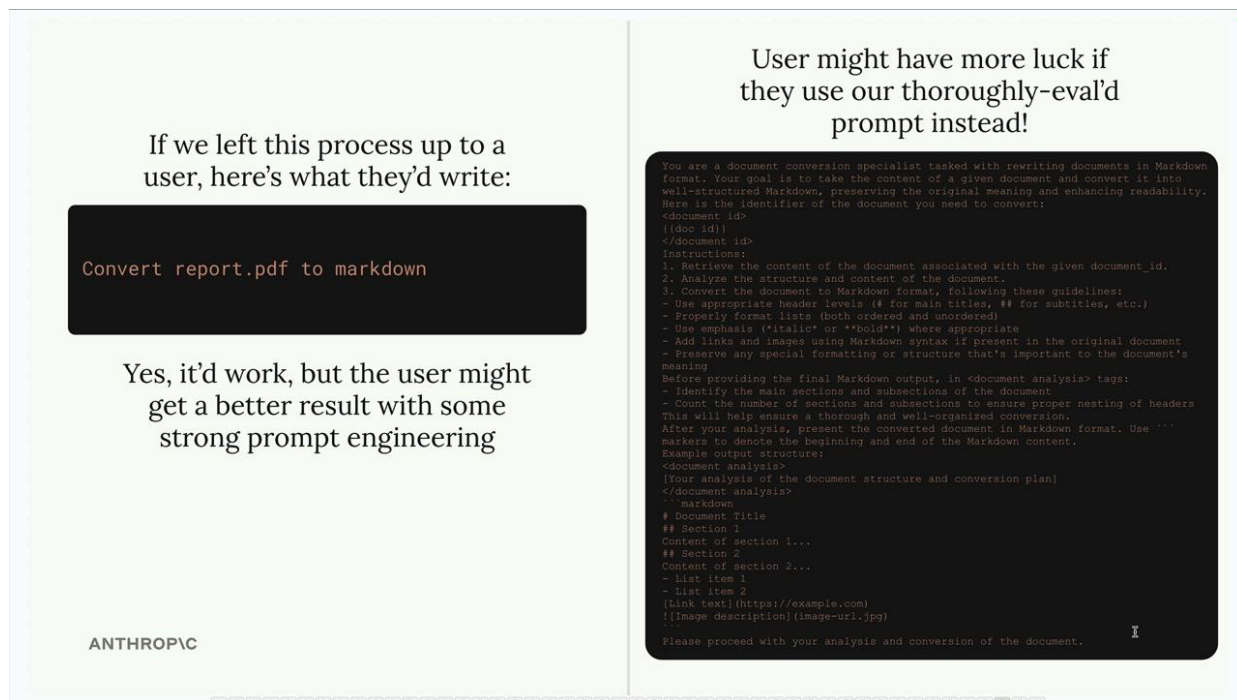
Ключова перевага в тому, що Claude отримує document content напряму в prompt, усуваючи потребу в tool calls для доступу до information. Це робить interactions швидшими й ефективнішими.

Integration with Your Application

Пам'ятайте, що MCP client code, який ви пишете, використовується іншими частинами application. Function `read_resource` стає building block, який інші components можуть викликати, щоб fetch document contents, list available resources або integrate resource data into prompts.

Такий separation of concerns зберігає code clean: MCP client обробляє communication із server, а application logic зосереджується на тому, як effectively use that data.

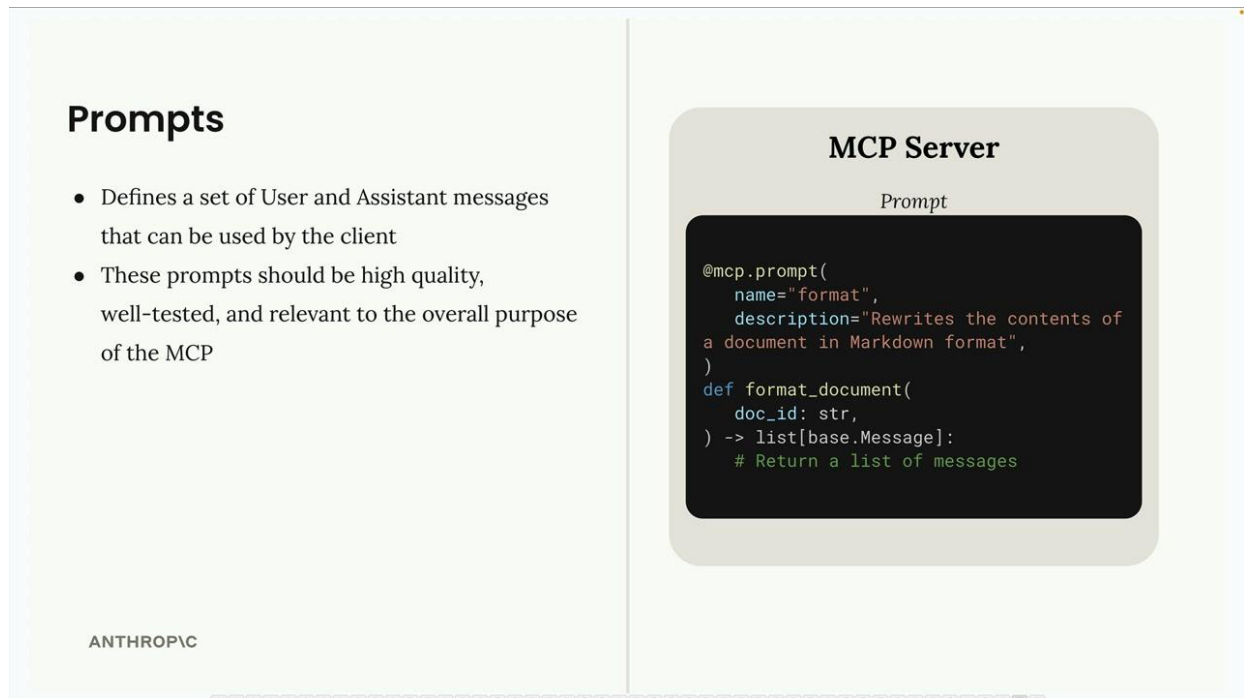
Defining prompts



Prompts у MCP servers дозволяють визначати pre-built, high-quality instructions, які clients можуть використовувати замість написання власних prompts from scratch. Думайте про них як про ретельно створені templates, що дають кращі results, ніж те, що users могли б сформулювати самостійно.

Why Use Prompts?

Припустімо, ви хочете, щоб Claude reformatted document into markdown. User може просто написати "convert report.pdf to markdown", і це спрацює. Але він, імовірно, отримає значно кращі results з thoroughly tested prompt, який містить specific instructions про formatting, structure і output requirements.



Ключова ідея в тому, що, хоча users можуть виконати ці tasks самостійно, вони отримають more consistent і higher-quality results, коли використовують prompts, які були ретельно developed і tested авторами MCP server.

How Prompts Work

Prompts визначають набір user і assistant messages, які clients можуть використовувати напряму. Коли client запитує prompt, ваш server повертає list of messages, які можна одразу надіслати Claude.

Basic structure виглядає так:

- Визначаєте prompts за допомогою decorator @mcp.prompt()
- Додавайте name і description для кожного prompt
- Повертаєте list of messages, які формують complete prompt

- Ці prompts мають бути high quality, well-tested і relevant to your MCP server's purpose

Building a Format Command

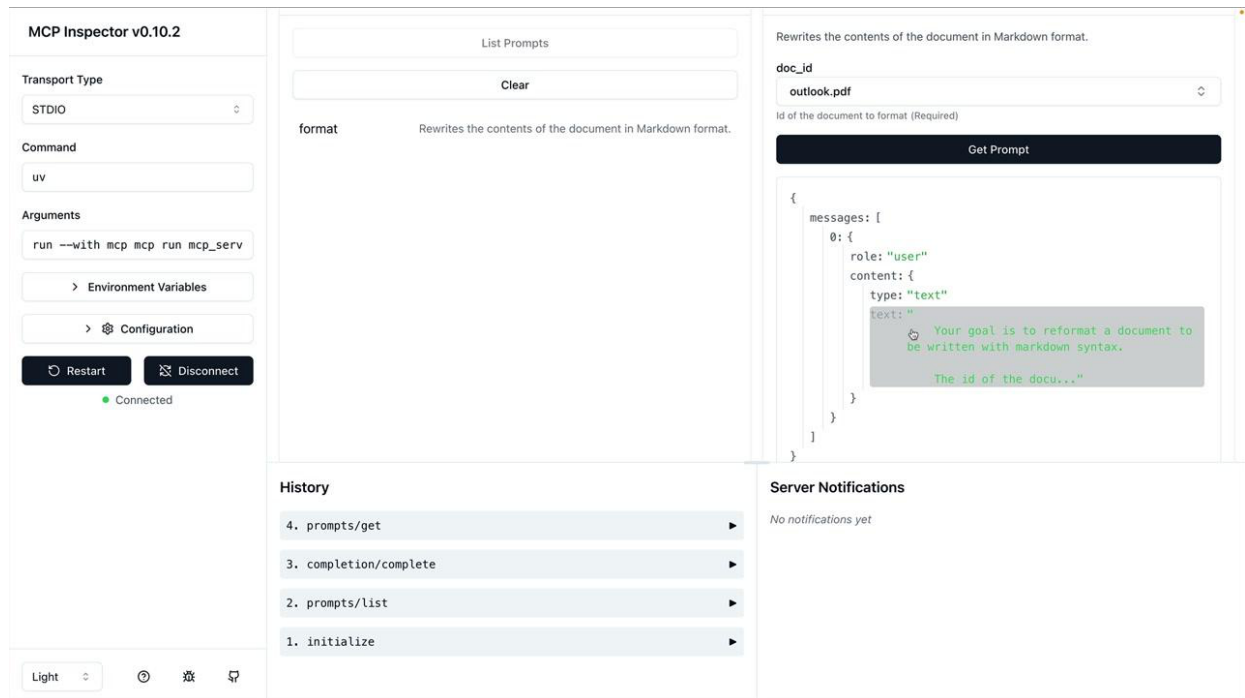
Ось як реалізувати document formatting prompt. Спершу потрібно import base message types:

```
from mcp.server.fastmcp import base
```

Потім визначте вашу prompt function:

```
@mcp.prompt(
    name="format",
    description="Rewrites the contents of the document in
    Markdown format."
)
def format_document(
    doc_id: str = Field(description="Id of the document to
    format")
) -> list[base.Message]:
    prompt = f"""
    Your goal is to reformat a document to be written with
    markdown syntax.
    The id of the document you need to reformat is:
    {doc_id}
    Add in headers, bullet points, tables, etc as necessary. Feel
    free to add in extra formatting.
    Use the 'edit_document' tool to edit the document. After the
    document has been reformatted...
    """

    return [
```



```
base.UserMessage(prompt)
]
```

Testing Your Prompts

Ви можете тестувати prompts за допомогою MCP Inspector. Перейдіть до section Prompts, виберіть prompt і надайте required parameters. Inspector покаже generated messages, які були б надіслані Claude.

Це дозволяє перевірити, що prompt correctly interpolates variables і produces expected message structure перед використанням у real application.

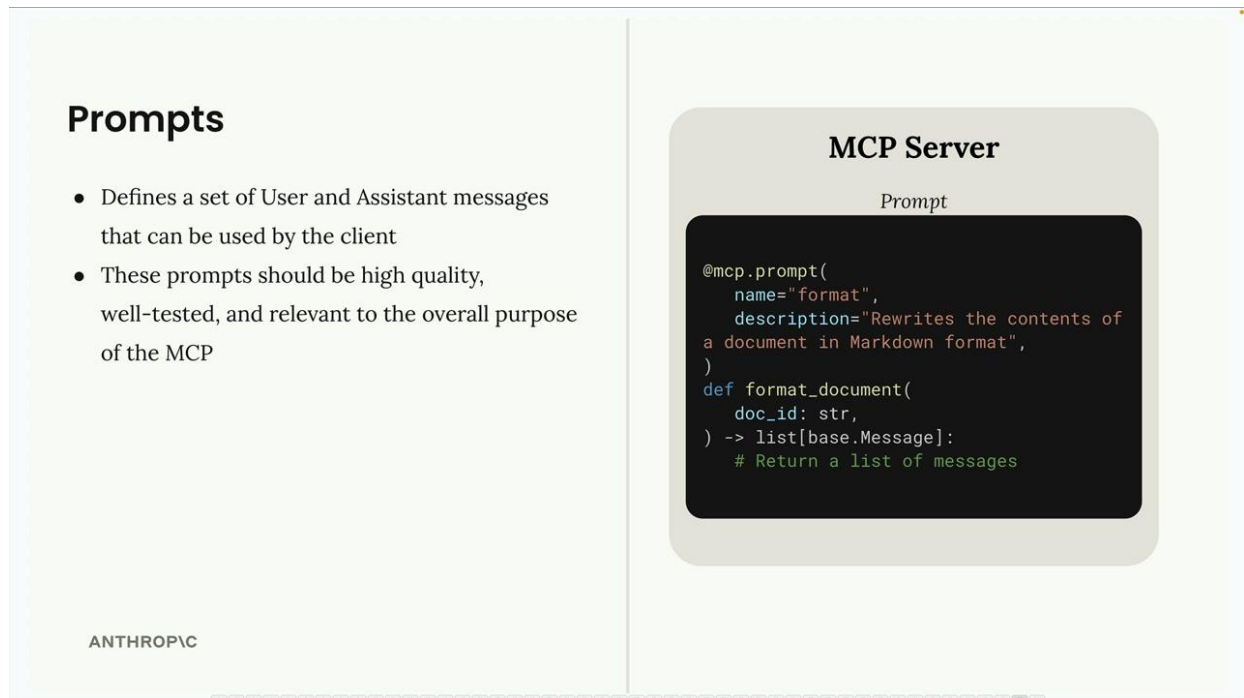
Best Practices

Під час створення prompts для MCP server:

- Зосереджуйтеся на tasks, які є central to your server's purpose
- Пишіть detailed, specific instructions замість vague requests
- Ретельно test prompts з різними inputs
- Додавайте clear descriptions, щоб users розуміли, що робить кожен prompt
- Враховуйте, як prompt працюватиме з tools і resources вашого server

Пам'ятайте, що prompts мають надавати value, яку users не могли б легко отримати самостійно - вони повинні представляти вашу expertise у domain, який охоплює ваш MCP server.

Prompts in the client



Prompts у MCP визначають набір user і assistant messages, які може використовувати client. Ці prompts мають бути high quality, well-tested і relevant to the overall purpose of the MCP server.

Implementing List Prompts

Перший крок - реалізувати method `list_prompts` у вашому MCP client. Цей method retrieves all available prompts from the server:

```
async def list_prompts(self) -> list[types.Prompt]:
    result = await self.session().list_prompts()
    return result.prompts
```

Ця проста implementation викликає session method `list_prompts` і повертає `prompts` array з result.

Getting Individual Prompts

Method `get_prompt` retrieves specific prompt з arguments, interpolated into it. Коли ви request a prompt, ви надаєте arguments, які передаються до prompt function як keyword arguments:

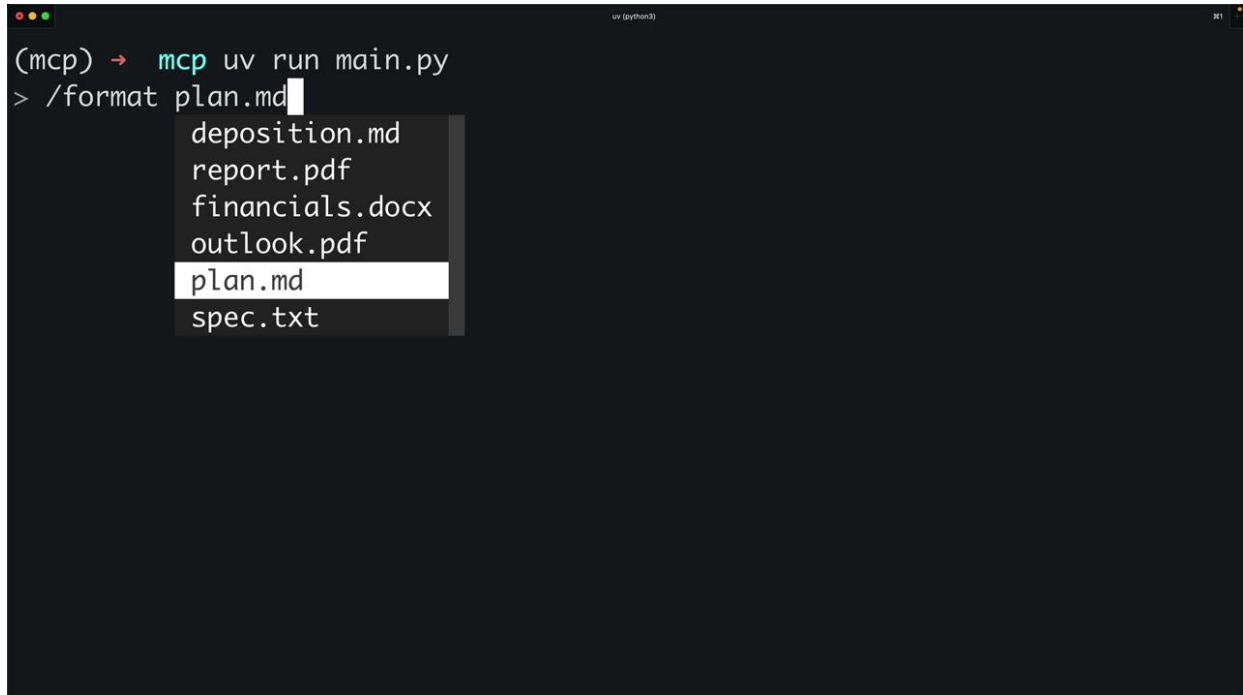
```
async def get_prompt(self, prompt_name, args: dict[str, str]):
    result = await self.session().get_prompt(prompt_name,
    args)
    return result.messages
```

Method повертає messages from the result, які формують conversation, що можна подати directly into Claude.

How Prompt Arguments Work

Коли ви визначаєте prompt function на server side, вона може accept parameters. Наприклад, document formatting prompt може очікувати parameter `doc_id`:

```
def format_document(doc_id: str):
    # The doc_id gets interpolated into the prompt
```



```

(mcp) → mcp uv run main.py
> /format plan.md
    deposition.md
    report.pdf
    financials.docx
    outlook.pdf
    plan.md
    spec.txt
  
```

Коли client викликає `get_prompt`, `arguments` dictionary має містити `expected keys`. MCP server передасть їх як `keyword arguments` до `prompt function`, дозволяючи вставити `dynamic content` у `prompt template`.

Testing Prompts in the CLI

Після `implementation` можна тестувати `prompts` через `command-line interface`. Коли ви вводите `forward slash`, `available prompts` з'являються як `commands`. Вибір `prompt` може попросити вас вибрати `available options` (наприклад, `document IDs`), а потім `complete prompt` надсилається Claude.

Workflow виглядає так:

1. User selects a prompt (наприклад, "format")

```

| Resource Type | Allocation | Timeline |
|-----|-----|-----|
| Development Team | 5 FTE | Phases 1-5 |
| QA Engineers | 2 FTE | Phases 3-4 |
| Project Management | 1 FTE | All Phases |
| Infrastructure | Cloud-based | Continuous |

## Risk Management

- Identified Risks:
  - Timeline delays
  - Scope creep
  - Technical challenges
  - Resource constraints

- Mitigation Strategies:

```

1. System prompts for required arguments (наприклад, який document format)
2. Prompt надсилається Claude з interpolated values
3. Claude може потім використовувати tools, щоб fetch additional data і complete task

Prompt Best Practices

Під час створення prompts для MCP server:

- Make them relevant to your server's purpose
- Test them thoroughly before deployment
- Use clear, specific instructions
- Design them to work well with your available tools
- Consider what arguments users will need to provide

Prompts bridge the gap між predefined functionality і dynamic user needs, даючи Claude structured starting points для complex tasks, водночас зберігаючи flexibility через parameterization.